

autofill-audit

Does a fifty kilobyte linear model get close enough to a twelve billion parameter one to be the right default for a developer tool?

A rule baseline, an n-gram model and a local language model, measured against each other on a seeded synthetic corpus of HTML forms

Olajide Badejo

2026-08-27

Abstract

A browser autofills a form field only when the markup gives it enough to recognise the field. **autofill-audit** loads a page in a real browser, walks the DOM including same origin frames and open shadow roots, classifies every control against the WHATWG autofill token set, and reports the controls that will not autofill together with the attribute to add. This report is the measurement half of that project.

Three classifiers are compared on the held out test split of a seeded synthetic corpus: a table of regular expressions, a calibrated character and word n-gram logistic regression exported to ONNX, and a twelve billion parameter instruction model at four bit quantization served locally. The comparison is paired, clustered at the template level, corrected across the whole family of comparisons, and gated on a practical effect size fixed before any of it ran.

The language model came third. It lost to both classical engines on the label weighted average, on the field weighted average, on every markup quality tier, and on the locales the classical engines had seen, and most of those margins survive correction. The one slice it led the n-gram model on is the held out locale, and that difference does not survive correction. The result that generalises furthest is not the accuracy gap, which depends on the corpus being synthetic, but the runtime inversion: for the classical engines the browser is essentially the whole wall clock and the classifier is a rounding error on a page load, and for the language model that reverses.

Half of the eight predictions registered before the benchmark were wrong, and the report says which and why. *Measured on a seeded synthetic corpus generated by this repository. Behaviour on real world pages is unmeasured and is stated as unmeasured.*

Contents

1	The problem, and why it is worth measuring	7
1.1	What breaks	7
1.2	The one design commitment everything else follows from	7
1.3	Why the output needs evidence rather than confidence	7
1.4	What this report is	8
1.5	How to read the numbers in this document	8
2	The taxonomy and where it comes from	9
2.1	Provenance	9
2.2	The growth rule, made mechanical	9
2.3	Coverage in the corpus as generated	10
2.4	Why not a bespoke ontology	10
3	The corpus, the generator, and the split	11
3.1	Why the corpus is generated	11
3.2	The composition, and the atom of the split	11
3.3	The markup quality tiers	12
3.4	The split policy and the leakage rule	12
3.5	The repair: a design that could not certify anything	13
4	The extractor and its blind spots	15
4.1	The seam that makes everything downstream testable	15
4.2	What is collected	15
4.3	Normalisation, and two bugs a property test found	16
4.4	The blind spots, stated as blind spots	16
4.5	What the phase turned out to be about	17
5	The classifier ladder	18
5.1	Why rules first	18
5.1.1	Ties are the safety mechanism, not a shortcoming	18
5.1.2	An honest dead rule	19
5.2	Why an n-gram logistic regression before a transformer	19
5.2.1	Features	19
5.2.2	The export route, and why the fallback was the design	20
5.2.3	A graph that cannot read its own weights	21
5.3	Calibration, and the part that got worse	21
5.4	The third rung, and what it is not	22
6	The audit engine: what gets said out loud	24
6.1	The decision procedure	24
6.2	Thresholds are per engine, and one of them is a mapping	25

6.3	The finding catalogue, in brief	26
6.4	A worked example	26
6.5	The exit code contract	28
7	Method	29
7.1	What one run is	29
7.2	Metrics	29
7.3	The statistical procedure	30
7.4	The dependency boundary, and what it cost	31
7.5	Reproducibility, and the one artefact that is marked dirty	31
8	The headline experiment	33
8.1	The answer, first	33
8.2	The two averages disagree, and the disagreement is the mechanism	34
8.3	The language model abstained least of all	35
8.4	Locale and tier	35
8.5	What a developer actually experiences	37
8.6	Latency, and the context that decides whether it matters	38
8.7	What the language model run did, and what it did not	39
8.8	The engine order in the benchmark was a methodological choice	40
8.9	The two classical engines reproduced their previous run exactly	40
8.10	The transformer that led the table and did not ship	40
9	Is the difference significant	42
9.1	What was run	42
9.2	The outcome categories, including the empty ones	42
9.3	What is certified, and what is not	43
9.4	One reading hazard, stated explicitly	43
9.5	The family size was fixed in advance, and it cost something	43
9.6	The harness's own verdicts, as a cross check	44
9.7	The secondary analysis, and why nothing cites it	44
10	What was predicted, and what was wrong	45
10.1	Why the predictions are here at all	45
10.2	Four held and four did not	45
10.3	The pattern across every phase of this project	46
10.4	The temptation that was registered against	47
11	Limitations	48
11.1	What the accuracy result does not license	48
11.2	The corpus is synthetic, and that is the load bearing limitation	48
11.3	Six locales is not multilingual	49
11.4	What the tool cannot see	49
11.5	The evaluation's own limitations	49
11.6	Development split evidence is one notch weaker	50
11.7	Class imbalance and small denominators	50
11.8	Cost was not measured because nothing was billed	50
12	Future work	51
12.1	Measure something real	51
12.2	Fix the double classification in the evaluation runner	51
12.3	Vary the one thing that was held fixed	51
12.4	A smaller encoder than the one that was measured	52

12.5	More templates per family	52
12.6	Close the cross repository tasks	52
12.7	Smaller things worth doing	53
12.8	What is deliberately not future work	53
A	The long tables	54
A.1	Every comparison in the family	54
A.2	The locale by tier grid	55
A.3	Per label behaviour	56
A.4	Confusions	57
A.5	Form family	58
A.6	Label coverage in the corpus	58
B	Provenance	60
B.1	Every result file this report cites	60
B.2	The result files that are here and are not cited	61
B.3	Version strings in the committed run logs	61
B.4	How to reproduce any number in this report	61

List of Tables

3.1	The realised generator grid and the partitions it was split into.	11
3.2	What widening the corpus did to the power of the design. More resamples would not have helped: at the original cluster count the test was already exhaustive. . .	13
3.3	The two classical engines before and after the corpus repair. These are not two measurements of the same thing: the test partition afterwards is a different set of templates. They are here because a moved metric owes an explanation, not because the columns are being averaged.	14
5.1	The training run that produced the shipped model.	19
5.2	The pre-registered hyperparameter sweep, on development macro-F1. The grid was registered before the model existed and was not widened afterwards.	20
5.3	The parity gate: the exported graph against the training time pipeline, on the full development split.	21
5.4	What calibration did to the model on the development split, including the summary statistic that moved the wrong way.	21
5.5	What each engine says about itself. A run log records this block, because a comparison table that named neither the model tag nor the quantization would not say what it had measured.	22
6.1	One threshold block per engine. An engine with no block is an error rather than an inheritance.	25
6.2	The n-gram engine's derived boundary and what it bought on the development split. The precision target was registered before the model existed and the derivation script was run unmodified.	26
6.3	The finding catalogue by severity. Each code carries a trigger, a named evidence list, a confidence or a documented tier, and a one line fix.	26
7.1	The pre-registered statistical policy and the power the realised design has. Every value in the upper half was fixed before the runs.	30
8.1	The headline benchmark: three engines on the test split, both averages, and the fourth row that does not exist.	33
8.2	Abstention, and accuracy on the fields each engine did commit to.	34
8.3	Seen locales against the held out locale. The unseen locale column is the multilingual claim's actual test: French forms whose templates are also unseen.	35
8.4	Macro-F1 by locale.	35
8.5	Macro-F1 by markup quality tier.	36
8.6	Finding level precision and recall. Precision is withheld for the two classical engines on the second code because neither made enough accusations to clear the reporting minimum, and that asymmetry is the finding rather than a formatting inconsistency.	37

8.7	Per field latency in microseconds, and the share of each run’s wall clock spent loading and extracting the page.	38
8.8	Where each run’s wall clock actually went, in seconds.	38
8.9	The language model run, including the failure mode that did not occur.	39
9.1	Every verdict category in the family of comparisons, including the two that came back empty. A summary listing only the categories that occurred would drop them on exactly the runs where nothing landed in them.	42
9.2	All comparisons in the family, read in the direction they were measured. The full table is Table A.2 in Appendix A; this one repeats its shape for reference.	43
10.1	The eight claims registered before the benchmark ran, the number that settled each one, and the outcome. The measured column is read from the result files by the table generator, so a prediction cannot be scored generously by whoever writes the report.	46
A.2	All comparisons in the corrected family, read in the direction they were measured. The <i>Comparison</i> column is built from the baseline, the candidate and the effect rather than from the comparison name, which reads backwards at a glance.	55
A.4	Macro-F1 for every locale and tier cell, all three engines. Every cell clears the reporting minimum, so nothing here is suppressed.	56
A.6	F1 per observed label per engine, with the test split support beside it. Read the rows against the support column: several labels are carried by one template in one position.	57
A.7	The largest confusions each engine makes on the test split.	57
A.8	The confusion pairs named in the specification before any measurement existed, and how often each actually occurred. Most of them never did, in either direction, for any engine.	58
A.9	Macro-F1 by form family.	58
A.11	Keyed fields per label across the whole generated corpus. This is the evidence for the growth rule’s clause that every label must be emitted by at least one answer key, and it is also the honest picture of how uneven the class distribution is.	59
B.2	Every run cited in this report, its engine, the commit its manifest names, and whether the working tree was clean when it was produced. A run marked dirty is not citable and the traceability job fails on a citation of one.	60

List of Figures

5.1	Reliability of the n-gram engine on the development split	22
8.1	Both averages for all three engines. They are shown together because they disagree about which classical engine leads and agree about which came last, and a chart carrying one of them would pick a winner by picking a denominator.	34
8.2	Macro-F1 by locale, with the held out locale shaded. It is the only column in which the language model leads the n-gram model, and it is also a column in which the rule table leads them both.	35
8.3	The tier profile, which is the most informative single picture in the benchmark. The two classical engines have strong and opposite slopes; the language model is nearly flat and below both until the tiers converge.	36
8.4	Where each engine lands on precision against recall for the missing declaration finding. This compares three threshold policies, not three classifiers. Each engine speaks under its own rule about when to speak, and the spread along the horizontal axis is mostly a picture of those rules.	37
8.5	Latency percentiles on a log axis, beside the share of wall time the browser takes. The log axis is not a stylistic choice: a linear one is unreadable across this range.	38

Chapter 1

The problem, and why it is worth measuring

1.1 What breaks

A browser fills a form field automatically only when the markup gives it enough to recognise what the field is for. When it cannot, the person on the other side of the screen types their street address by hand, on a phone, at the last step of a checkout. That is not a cosmetic defect. It is the point in a funnel where people stop.

The web platform already says how to avoid it. The WHATWG HTML Living Standard defines the `autocomplete` attribute and a fixed set of autofill field name tokens, and the browser vendors' own form guidance explains what their heuristics do with them [17, 4, 5]. The hard part is not knowing the rule. The hard part is standing in front of a page somebody else wrote, with sixty controls on it, and working out which ones are broken and exactly what to add to each.

That is the gap this tool sits in. It loads a page in a real browser, walks the document including same origin frames and open shadow roots, collects the signals a browser's own heuristics would use, classifies every control against the specification's token set, and reports the controls that will not autofill together with the attribute to add to each one.

1.2 The one design commitment everything else follows from

The label space *is* the specification's token set. Not a bespoke ontology that is later mapped onto it.

This is worth stating early because it removes an entire class of bug from the product. A control classified as `postal-code` produces the advice `add autocomplete="postal-code"`, and there is no second, undertested translation layer between the prediction and the fix. A project that invented its own categories would need a mapping table from category to token, that table would be wrong somewhere, and the wrongness would surface as confident bad advice on somebody's production markup.

1.3 Why the output needs evidence rather than confidence

The output of this tool is a list of accusations. *This field will not autofill, and here is why.* A developer acts on those, edits production markup, and ships.

A wrong finding costs them time and costs the tool its credibility. An unattributed finding costs more than either, because a finding that says *this looks like a postal code* without saying which signal fired and how sure the classifier is cannot be argued with, which means it cannot be checked, which means the only available responses are to trust it or to delete the check.

So every finding this tool emits carries a `signals` list naming the concrete inputs that triggered it, and, from the point at which a probabilistic classifier exists, a calibrated confidence. Below the reported threshold the tool says `LOW_CONFIDENCE` or `UNDETECTABLE_FIELD` rather than guessing quietly. There is no third branch, and in particular there is no path by which a rule table's default case becomes a confident answer.

1.4 What this report is

The tool is one half of the project. The other half is the question of whether any of it works, and that question has a specific form here.

Does a fifty kilobyte linear model that runs in microseconds on a CPU get close enough to a twelve billion parameter language model to be the right default for a developer tool?

That is the headline experiment. It is a product question rather than a research one, and it was chosen because it is the question an engineer actually faces when deciding what to put in a wheel that people install with one command. This report answers it with numbers, and prints the answer the measurement gave rather than the one the project was built expecting.

The measurement itself is described in Chapter 7, the numbers are in Chapter 8, the significance procedure and its verdicts are in Chapter 9, and the predictions that were registered before any of it ran are scored in Chapter 10. What the answer does not license is in Chapter 11, and it is a longer chapter than the answer.

1.5 How to read the numbers in this document

Measured on a seeded synthetic corpus generated by this repository. Behaviour on real world pages is unmeasured and is stated as unmeasured.

Two properties of this document are worth knowing before the first table.

No number here was typed by hand. Every table is generated from a committed result file by `scripts/make_report_tables.py`, every figure by `scripts/make_report_figures.py`, and every number quoted in a sentence is a macro defined by the first of those beside the path it was read from. The chapters of this report contain no digits at all. A continuous integration job scans these sources for anything shaped like a measurement and fails the build unless the line names the result file behind it, then follows that file to its run manifest and the manifest to a commit object this repository can produce.

Every result cited here was produced from a clean working tree. A run taken from a tree with uncommitted changes is marked dirty in its manifest and is not citable, and the traceability job fails on a citation of one. One artefact in this repository does carry that mark, and Chapter 7 says which and what follows from it, rather than leaving the reader to find it.

Chapter 2

The taxonomy and where it comes from

2.1 Provenance

The ground truth label space is the WHATWG autofill field name token set [17] plus a small, enumerated set of extra labels. Nothing in it was invented for convenience, and the extras are named individually rather than admitted by a rule, because a category that can be added by a rule is a category that grows until the label space is nobody's specification.

The extras exist for outcomes the token set has no word for, and each one earns its place by naming something a report has to be able to say:

- `NOT_AUTOFILLABLE`, for a control that is a real control and has no autofill meaning at all: a search box, a consent checkbox, a quantity stepper. Without it the only honest answer for such a field would be `UNKNOWN`, which reads as *the tool could not tell* when the truth is *there is nothing to tell*.
- `UNKNOWN`, for a control whose evidence genuinely does not decide. This is an answer, not an absence of one.
- `COMPOSITE_UNSPPLIT`, for one input holding what the specification models as several fields, the archetype being a single expiry box accepting month and year together.
- `CC_EXP_SPLIT_MONTH` and `CC_EXP_SPLIT_YEAR`, for the two halves of a split expiry pair, which are individually meaningless and jointly a `cc-exp`.

2.2 The growth rule, made mechanical

A label may exist only if it is a specification token or an enumerated extra, *and* is emitted by at least one corpus answer key, *and* is reachable by at least one rule or present in the model's training distribution, *and* is asserted by at least one test. A label that cannot satisfy all four is deleted, or carries a dated and tracked exemption.

That is a rule somebody will forget, so it is a script. `scripts/check_reachability.py` runs in continuous integration, reads the taxonomy from the one source file that defines it, and fails the build on any clause it can currently enforce. Its clauses were activated one at a time as the phases that made each enforceable landed, and each activation moved a clause out of the file's own pending list in the same commit that made it checkable, which is the only way a pending list stays honest.

Two further properties of the check are worth naming because they are strengthenings rather than restatements.

It scans for label strings written outside the taxonomy module. Ground rule 6 says the taxonomy has exactly one definition in code. The check makes that mechanical by walking the source and the scripts for label literals. It deliberately excludes the handful of tokens that are also ordinary English words, because a check that fires on the word `name` in a comment is a check that gets switched off within a week.

It requires a training floor. Every label a model can predict must appear in the training partition with at least a full template’s worth of rows. That clause was added at the corpus repair described in Chapter 3, after three labels were found sitting at zero training rows, which means the model had been asked about classes it had never once seen and could only ever get wrong.

2.3 Coverage in the corpus as generated

Every label in the taxonomy is emitted by at least one answer key in the generated corpus. Table A.11 in Appendix A lists the keyed field count per label, which is the evidence for clause (b) of the growth rule and also the honest picture of how uneven the class distribution is. Several labels are carried by a single template in a single position, and the per label results in Table A.6 should be read against that column rather than on their own.

2.4 Why not a bespoke ontology

The alternative was tempting and is worth refusing in writing.

A bespoke ontology would have let the corpus, the rules and the model agree with each other perfectly, because all three would have been designed together. It would also have made the tool’s central output, the exact attribute value to add, a translation step rather than a formatting step, and every disagreement between the ontology and the specification would have become a silent wrong answer in production markup.

More importantly, it would have made the evaluation meaningless outside this repository. Measuring a classifier against categories the same project invented measures internal consistency. Measuring it against the token set that browsers actually implement measures the thing the tool claims to do.

Chapter 3

The corpus, the generator, and the split

3.1 Why the corpus is generated

No script in this repository fetches HTML from a live site and writes it to disk as training data. Real world *structure* was observed and reimplemented; real world *content* never enters the repository, including the author’s own. Card fields use the officially published test numbers [15], and names and addresses are self evidently invented.

That is a privacy commitment, and it is also what makes the measurement reproducible: a corpus that is a deterministic function of a seed can be regenerated byte for byte by anyone who has the seed, and two corpora with the same manifest digest are the same corpus. The generator is checked by running it twice and diffing the output, which is the phase gate that produced it.

3.2 The composition, and the atom of the split

Table 3.1: The realised generator grid and the partitions it was split into.

Corpus quantity	Value
Families	5
Templates	40
Locales	6
Markup quality tiers	4
Variants per cell	1
Forms generated	960
Keyed fields	10351
Generator seed	20260825
Base year	2026
Train forms	500
Dev forms	120
Test forms	240
Excluded forms	100
Test templates	10

Source: `corpus/manifest.json`

Source: `models/dev_metrics.json`

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Five form families, several structurally distinct templates each, instantiated across six locales

and four markup quality tiers. Table 3.1 gives the realised counts.

The template is the atom of the split, so the template had to become the atom of the generator. A template asks for a name block or an address block *by style*; the locale profile decides which slots that style expands to and in what order. A template that listed address line one and address line two directly would silently assert that every locale composes an address the same way, and the multilingual claim this project makes would then be a claim about one language written six times.

The locale profiles are the substance of that. Each carries label text, placeholder text and identifier strings; which slots exist; the order they appear in; and how personal names decompose. German and French address blocks carry no administrative area field at all. Japanese leads with the postal code and adds a kana name pair. Nigerian postal codes are frequently absent, expressed as a presence probability rather than a flat absence, because *frequently* is the honest shape of that fact.

One expectation was wrong and the correction is instructive. The plan anticipated hand authored fallback tables for the Nigerian locale on the assumption that the value provider library would be thin there [6]. It was not: working providers existed for every locale, including the kana name providers Japanese needs. The hand authored tables still exist and still matter, but for a different reason: a value provider generates *values*, and no provider knows that a German form labels a postal code field *PLZ* and names the input *plz*. Interface text was always going to be hand written, and that is what a locale profile is.

3.3 The markup quality tiers

The four tiers are the axis the whole tool is aimed at, so they are graded rather than binary.

clean Correct labels, correct identifiers, correct declarations. A correct page must produce silence, and that is asserted over the whole clean slice rather than sampled.

partial Correct structure, missing declarations. The ordinary case.

mixed Sections written to different standards inside one form, which is what a real page looks like after two teams and three years.

hostile Labels stripped, identifiers obfuscated, declarations absent or wrong.

The hostile tier turned out to be the most interesting to design. Half its controls carry the label text as a placeholder, which is the placeholder as label antipattern and is genuinely classifiable. The other half carry no text signal at all and leave only the identifier, which is where the cross locale claim is actually tested, because the identifiers are localised. One control per form is deliberately undeterminable, and predicting UNKNOWN on it is the correct answer rather than a failure. If every hostile control were equally hopeless, the tier would measure nothing except that the tier is hard.

3.4 The split policy and the leakage rule

Whole templates are assigned to partitions, never individual forms. Two forms of one template in two locales share an author, a naming convention and a slot order, and splitting between them would measure how well a classifier memorises one author rather than how well it generalises.

One locale is held out entirely. French appears in no training form. That gives the report a genuine unseen locale column rather than a claim that character n-grams ought to transfer.

Those two rules interact in a way the specification left open, and the resolution is worth recording.

A French form whose template is in the training partition cannot go to training, because the locale is held out, and must not go to development or test, because its template is a training template and that would leak the convention the split exists to separate. Such forms go to a fourth partition, **excluded**: generated, recorded, and used by nothing. The reported unseen locale slice is the French forms whose templates are also unseen, which is clean on both axes.

3.5 The repair: a design that could not certify anything

The most important thing that happened to this corpus was that it was found to be too small in a specific, arithmetic way, and was rebuilt before the headline benchmark rather than after it.

The significance test resamples whole templates, because fields inside a template share an author. The first realised grid put one template per family in the test partition, so the test split held 5 templates. A paired sign flip permutation over that many clusters has 32 arrangements, so the smallest two sided p value the design can produce is 0.0625, and the pre-registered false discovery level is 0.05.

The design could not reach its own level, at any effect size. All 11 comparisons in the first analysis came back inconclusive for that reason, several of them sitting exactly at the floor, meaning the observed difference was more extreme than every other arrangement the design permits and was still not significance.

Table 3.2: What widening the corpus did to the power of the design. More resamples would not have helped: at the original cluster count the test was already exhaustive.

Design quantity	first	repaired	headline
Clusters	5	10	10
Arrangements	32	1024	1024
Smallest attainable p	0.062500	0.001953	0.001953
Comparison family size	11	11	33
Inconclusive: cannot reach alpha	11	0	0

Source: `experiments/results/analysis/2026-08-26T02-53-59Z_analysis_336175f/analysis.json`
`experiments/results/analysis/2026-08-26T06-23-41Z_p5r-analysis_0d900ff/analysis.json`
`experiments/results/analysis/2026-08-26T21-20-11Z_p6-analysis_6457ac7/analysis.json`

Three options were available: widen the corpus, accept the floor and report every comparison as inconclusive forever, or pre-register a different alpha. The third was refused on sight, because choosing a level after discovering that the original one was unreachable is indistinguishable from moving the goalposts. The second leaves the headline experiment answering its question with effect sizes and no significance at all. The first was taken.

The cost was real and was paid in the open: a regeneration, a retraining, a threshold rederivation and a re-measurement. Every recorded number in the repository moved at once, and the explanation was committed *before* the work rather than after it, because a phase that is about to move every metric it has owes the reader the reason first.

Table 3.3 is that explanation. The rule baseline barely moved, because it is a fixed function of the markup and a different set of templates simply asks it slightly different questions. The n-gram model moved substantially, because it gained ten training templates and lost the three classes it had been asked about without ever having seen them.

What deliberately did not move: the seed, the held out locale, the excluded partition rule, the alpha, the false discovery rate, the absolute practical effect threshold, the clustering unit, and the precision target behind the decision threshold. Repairing a corpus is only honest if the standards

Table 3.3: The two classical engines before and after the corpus repair. These are not two measurements of the same thing: the test partition afterwards is a different set of templates. They are here because a moved metric owes an explanation, not because the columns are being averaged.

Metric	rules before	rules after	ngram before	ngram after
macro-F1	0.7785	0.7911	0.4151	0.7394
micro-F1	0.7744	0.7678	0.7041	0.8226
macro-F1, unseen locale	0.7193	0.7276	0.4057	0.5637
macro-F1, clean tier	0.8500	0.8677	0.5885	0.9350
macro-F1, hostile tier	0.5163	0.4681	0.4052	0.5409
Abstention rate	0.2676	0.2620	0.0508	0.0363
Accuracy when committed	0.9773	0.9702	0.6996	0.8204

Source: `experiments/results/test/2026-08-26T02-50-54Z_rules_57d2aed/metrics.json`

Source: `experiments/results/test/2026-08-26T06-18-34Z_p5r-rules_0d900ff/metrics.json`

Source: `experiments/results/test/2026-08-26T02-52-08Z_ngram_57d2aed/metrics.json`

Source: `experiments/results/test/2026-08-26T06-21-03Z_p5r-ngram_0d900ff/metrics.json`

it is measured against stay exactly where they were, and a phase that moved a threshold and a corpus at the same time would have produced two changes and no attribution.

Chapter 4

The extractor and its blind spots

4.1 The seam that makes everything downstream testable

The extractor turns a page into a list of field descriptors, and that list is the only interface anything downstream ever sees. A classifier never sees the DOM and never sees an answer key. It sees a descriptor.

The implementation splits at exactly that boundary, and it was the single most useful decision of the phase that built it. An in page traversal script reads the document and returns plain records. Everything after that is ordinary Python over those records, and the function that turns records into descriptors takes the same input a browser would have produced and does the rest with no browser at all.

The consequence is that selector generation, group detection, the honeypot rule, declaration parsing and every normalisation step are covered by tests that run in milliseconds, and the browser driven tests are left to prove the one thing only a browser can prove: that a real page does say what the records claim. It also means the coverage gate does not rest on a browser being installed, which matters the first time a browser automation upgrade goes sideways [13].

4.2 What is collected

Per control: the identifier and name, every label source in a documented precedence order, placeholder, ARIA attributes [16], input type, pattern, length constraints, required and readonly flags, option lists for selects, the declared `autocomplete` value parsed into its token grammar, the surrounding heading and legend context, the frame and shadow root the control lives in, and a stable selector.

Two collection decisions are worth naming.

Option lists are truncated at twice the largest list this project reasons about. The descriptor has no field for a total option count, so the only way to tell a complete list of twelve months from the first twelve of five thousand is to put the cut where a meaningful list never reaches.

Several signals are collected and deliberately stop before the descriptor. They are collected because the signal specification says to and they stop because the descriptor is a fixed downstream contract that every golden test inherits. One of them, whether a select accepts multiple values, is genuinely load bearing: a multi select with twelve options is a month picker to a careless detector and a multiple choice control to a careful one.

4.3 Normalisation, and two bugs a property test found

Text normalisation is unglamorous and turned out to be where the only genuinely subtle defects of the extractor lived. The published step order is compatibility normalisation, case folding, camel case splitting, delimiter splitting, then a stoplist. Taken literally that order destroys its own worked example: folding the whole string before looking for lower to upper transitions turns `firstName` into `firstname`, in which no boundary exists. The implemented reading inserts boundaries first and folds each resulting token, which loses nothing the fold was there for, because both properties it provides are per character.

Idempotence is asserted as a property, and it earned its place twice.

Combining marks were being treated as separators. The splitter was built on the language’s word character class, which excludes combining marks. The failing case found by the property test was the Turkish dotted capital I: folding it yields a letter followed by a combining dot, splitting on that dot drops the dot, and a second pass over the same text returns something different from the first. The interesting part is what else the bug broke. Splitting on combining marks shreds Devanagari, Thai, pointed Hebrew and anything in decomposed form. It would never have shown up in this project’s six locales, and it would have been waiting for the first person to point the tool at a page outside them.

Case folding does not always produce lowercase. Cherokee folds upward, so folding a lowercase Latin letter beside a Cherokee capital leaves a case transition that the camel case pass, which runs before the fold, never saw. Feed the output back in and it splits differently. The fix is to bracket the fold with boundary insertion rather than to run it once before, which is the only order that satisfies both constraints.

The reaction to the first counterexample matters more than the fix. The first instinct was to narrow the property to the six locales the corpus covers and call idempotence a practical guarantee. Both counterexamples were outside those locales and both were real defects, so the strategy went the other way instead and now sweeps the whole basic multilingual plane, each codepoint alone and beside four neighbours in both orders.

4.4 The blind spots, stated as blind spots

These are the cases where the tool cannot see, and each produces a named outcome rather than silence.

Closed shadow roots. Unreachable by design of the platform, for this tool and for every other script on the page. Reported as `UNDETECTABLE_FIELD`, with the reason recorded and a fix that suggests exposing the control in light DOM or declaring the attribute on the host.

Cross origin frames. The frame is detected, its contents are not read, and the finding says that a hosted payment field is a common and correct pattern rather than implying a defect. The accessibility test is made from inside the parent document by asking for the frame’s document in a guarded block, which is exactly the check a script on that page would make.

Canvas rendered widgets. A canvas is always recorded as a region. A page level undetectable descriptor is emitted only when the page yielded no controls at all, because emitting one per large canvas would make every hostile checkout report one more field than its answer key holds and the reconciliation would never balance.

Multi step forms. Only the step currently in the document is audited. There is no click through, because clicking is filling.

Whether the browser actually autofills. The tool verifies that the markup gives the browser what it needs. It does not verify the browser’s behaviour, and it never claims to.

One reconciliation discrepancy is worth recording because it looks like a defect and is not. The extractor returns document order; an answer key lists fields in the generator's slot order, which appends a tier's extra controls to the end. Those agree everywhere except the mixed tier, where a hostile block sits in the middle of a form whose extra controls belong to that block. The selectors are the contract; the order of the list is not, and the property that every such form is on the mixed tier is asserted rather than assumed.

4.5 What the phase turned out to be about

The DOM walk itself is a few dozen lines and was right almost immediately. What took the time was deciding what counts as a honeypot, which label source wins, when a canvas counts as a field, whether two adjacent selects are an expiry pair, and what *cannot be accessed* means for a frame.

Every one of those is a judgement that ends up in a report somebody reads, and none of them is discoverable from the platform API. The corollary is that the tests worth having are the ones that pin the judgements, not the ones that pin the traversal, and the test suite is shaped accordingly.

Chapter 5

The classifier ladder

The ladder is fixed in build order and was not reordered: a rule table, then an n-gram logistic regression, then, only if the measured ceiling justified it, a language model. The ordering is a claim about what a project owes its users, and it is defended here rather than assumed.

5.1 Why rules first

A rule table is a table of regular expressions with a tier and a signal name on every row. It has no probabilities and does not pretend to. It is also the version of the tool that is genuinely useful to somebody who just wants their checkout page fixed, and it shipped before any machine learning existed.

That ordering means the project can answer a question most projects cannot: is the model solving a problem anybody has? A project that ships the model first and the product last has no baseline to answer it with, and every subsequent measurement is a comparison against nothing.

5.1.1 Ties are the safety mechanism, not a shortcoming

The rule engine has six precedence tiers. A tier decides when it yields a unique match and otherwise falls through, and a tie surviving every tier is UNKNOWN. That reads like a tidy up clause. It is the single most load bearing rule in the engine.

German writes *Straße und Hausnummer* for both a whole street address and the first of several address lines. French writes *Adresse* for both. A field labelled only *Password* is a login field on one page and a registration field on the next. In each of those the wording genuinely does not distinguish two labels, and the correct answer is that the tool does not know.

Once that is accepted the table changes shape. Instead of writing one pattern per label and hoping they do not overlap, the overlaps become deliberate: both labels carry the same phrase at the same weight, the tier ties by construction, and the engine falls through to a tier that might separate them. The property that a correct page produces no finding above the lowest severity fell out of that design rather than being tuned into existence, and it passed the first time it was run across the entire correct markup slice.

The alternative would have been to break each tie by picking a favourite. That produces a confident instruction to write the wrong token into somebody's production markup, and it would have looked like better coverage on every metric except the one that matters.

5.1.2 An honest dead rule

The specification names the Japanese postal mark among the strings the postal code label should match. It cannot fire: normalisation treats a lone symbol character as a delimiter, so the mark never reaches a token stream, and the Japanese postal field is reached through its word form instead.

The rule is in the table anyway, with a comment saying it cannot fire and why, and a test asserting both halves of that. Making it reachable means changing what counts as a token character, which changes every golden snapshot and every committed fixture expectation. That is a deliberate decision with a cost rather than a quiet fix, and it is recorded as one.

5.2 Why an n-gram logistic regression before a transformer

Three reasons, in the order they matter.

It fits in a wheel. The exported model plus its vocabulary and calibration is small enough to install without comment, and inference needs an array library and a runtime and nothing else. A tool that requires a large download or a GPU to audit a form is a tool nobody installs.

It is inspectable. A linear model’s weights name the character and word n-grams behind a prediction, which is exactly what law 1 requires a finding to carry. That property is not incidental to the design; it is why the rung exists at that height.

It sets a floor that a bigger model has to beat. Without it, any transformer result is a comparison against a rule table alone, and the interesting question, whether the extra capacity buys anything on a task whose whole input is a short list of attribute strings, never gets asked.

5.2.1 Features

Five blocks: character n-grams over the normalised token stream, word n-grams over the same, one hot categorical signals, option shape booleans, and structural buckets. Table 5.1 gives the realised widths.

Table 5.1: The training run that produced the shipped model.

Training quantity	Value
Train forms	500
Train rows	5402
Dev forms	120
Dev rows	1596
Excluded forms	100
Classes fitted	42
Feature width	7986
Character n-gram features	5760
Word n-gram features	2176
Categorical features	32
Option shape features	5
Structural features	13
Chosen inverse regularisation	8.0000
ONNX opset	26
Dev macro-F1	0.7866
Dev accuracy	0.8421

Source: `models/dev_metrics.json`

Source: `models/train_manifest.json`

Table 5.2: The pre-registered hyperparameter sweep, on development macro-F1. The grid was registered before the model existed and was not widened afterwards.

Inverse regularisation	Dev macro-F1
0.25	0.7308
0.50	0.7375
1.00	0.7505
2.00	0.7612
4.00	0.7660
8.00	0.7690
chosen	8.00

Source: `models/dev_metrics.json`

The sweep chose the edge of its own grid. Table 5.2 shows the chosen value sitting at the largest value the grid offered. The grid was pre-registered, so it was not widened afterwards: widening a pre-registered grid because the answer landed at its edge is how a sweep becomes a search for a number. The consequence is recorded instead, in the model card and here, and it is that this model may be less regularised than a wider grid would have chosen. A later phase that revisits it re-registers first.

5.2.2 The export route, and why the fallback was the design

The plan anticipated a fight with the converter for text vectorizers [10] and named a fallback: move the whole text pipeline into hand written code, export only the linear layer, and take that quickly rather than spending a day coaxing a converter.

The fight never started, because the feature set was never convertible. Three of the five feature blocks are categorical one hots, option shape booleans and structural buckets, which no standard transformer produces. Putting the whole featurisation in the graph would have meant writing custom converters for three bespoke transformers *and* trusting the vectorizer converters, which is a larger version of exactly the risk the fallback exists to avoid.

So the fallback’s hand written featuriser is not a retreat from the design. It is the only design in which one implementation serves both training and inference, which the feature specification required from the beginning. The second argument arrived for free and will matter longer: a vectorizer anywhere in the inference path puts a full machine learning framework on the dependency list of a tool people install with one command [14, 11].

The cost is real and is tested rather than waved past. A hand written analyser is slower than a compiled one, and its faithfulness to the reference analyser is a claim rather than an identity, so the claim is asserted character for character over inputs including empty strings, words shorter than the n-gram length, and Japanese text. The detail an independent reimplementer gets wrong is the short word rule: a word shorter than the n-gram length yields its padded self once rather than once per length, and getting that wrong silently triples the weight of every two letter token.

Table 5.3 is the gate that makes the export believable. The opset was chosen by probing downward from the newest the installed toolchain defines and taking the first that both converts and opens in a session; the converter warns at every attempt that the requested opset is above the newest it has been tested against. That warning is recorded rather than filtered, because it is precisely the case where a successful export proves nothing and only the parity gate proves something.

Table 5.3: The parity gate: the exported graph against the training time pipeline, on the full development split.

Parity quantity	Value
Dev rows compared	1596
Argmax agreements	1596
Agreement rate	1.0000
Largest absolute probability delta	0.000000477
Tolerance	0.000010000
Passed	yes

Source: `models/dev_metrics.json`

5.2.3 A graph that cannot read its own weights

Worth recording because it changed a design. The exported graph holds its coefficients, intercepts and class order in a node’s *attributes* rather than in graph initialisers, and the runtime does not hand a caller a node’s attributes. So the inference path cannot read its own weights, and naming the n-grams behind a prediction would otherwise have required the full model format library at runtime to produce one line of a report.

The resolution is a separate evidence table, written by the same training run, holding each class’s highest weighted features. A derived file is a drift risk, so the parity suite reads the weights back out of the graph and asserts the table agrees with them. A stale evidence file now fails the build rather than naming the wrong n-grams in a finding, which would have satisfied law 1 in form and violated it in substance.

5.3 Calibration, and the part that got worse

Table 5.4: What calibration did to the model on the development split, including the summary statistic that moved the wrong way.

Calibration quantity	Value
Classes fitted	42
Isotonic switchover, dev positives	100
Classes calibrated by identity	2
Classes calibrated by isotonic	1
Classes calibrated by sigmoid	39
Expected calibration error before	0.0281
Expected calibration error after	0.0454
Dev macro-F1 before calibration	0.7690
Dev macro-F1 after calibration	0.7866

Source: `models/dev_metrics.json`

Expected calibration error was *lower before calibration than after it* (Table 5.4, Figure 5.1). The uncalibrated model on this corpus is already close to honest, and refitting a one versus rest sigmoid per class on a development split of one template per family, then renormalising, moves classes that were already well behaved.

The calibrator stayed, for two reasons that are both stated rather than assumed. Macro-F1 is higher with it. And the argument for calibrating at all is about the number a single finding quotes to a developer, not about an average gap across a split. What changed is the model card, which now says the error went the wrong way and by how much. Dropping the calibrator because its summary statistic got worse, without re-registering, would be tuning on the thing being measured.

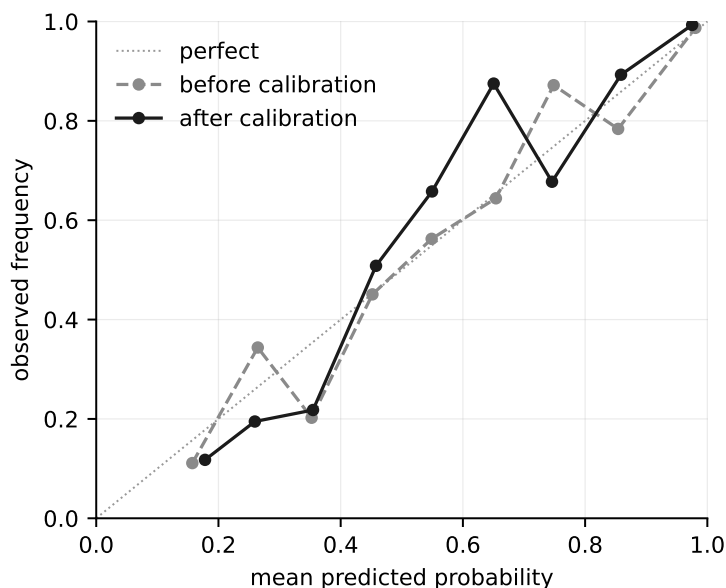


Figure 5.1: Reliability of the n-gram engine on the development split, before and after calibration. Only this engine appears: the rule baseline has no probability at all, and the language model’s confidence is self reported, so neither belongs on an axis of calibrated probability. Generated by `scripts/make_report_figures.py` from `models/dev_metrics.json`.

One class reached the isotonic switchover, the label covering search boxes and consent checkboxes. Every other class is calibrated by a sigmoid, and a handful with no development positives at all are uncalibrated and named as such in the card.

5.4 The third rung, and what it is not

The third engine is a twelve billion parameter instruction model at four bit quantization, served locally through a runtime that enforces a JSON response schema at decode time [9]. It is optional at runtime, never required for an audit, and never exercised by continuous integration.

Table 5.5: What each engine says about itself. A run log records this block, because a comparison table that named neither the model tag nor the quantization would not say what it had measured.

Engine property	Value
rules confidence kind	tier
ngram confidence kind	calibrated-probability
ngram execution provider	CPUExecutionProvider
ngram ONNX opset	26
llm confidence kind	self-reported
llm model tag	mistral-nemo:12b-instruct-2407-q4_K_M
llm quantization	q4_K_M
llm temperature	0.0
llm batch cap	40
llm structured output	json_schema passed with every request
llm prompt version	1.0.0

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Three design points decide whether the comparison measures anything at all.

The declaration is dropped from what the model sees. Leaving it in lets the model read the answer off the page on every clean tier form. The obvious check, a substring search for the

declared token in the serialised payload, is wrong in both directions: a field named `email` that also declares `autocomplete="email"` would trip it, and that combination is common enough in a clean corpus that the check would have fired on the first real run and been switched off, while a declaration reaching the payload through a derived value would not trip it at all. So the property checked is *independence*: pruning a descriptor and pruning the same descriptor with its declaration stripped must produce identical payloads. That has neither failure mode and is asserted on every request rather than reviewed once.

The keep list is a floor, not a ceiling. The specification's list of what pruning keeps omits several signals the featuriser feeds to the n-gram model. Withholding a signal one engine has from the other tilts the comparison exactly as far as handing over the whole document would, in the opposite direction and with a better conscience. The rule applied is that the language model sees what the featuriser sees and nothing more.

Its threshold block is zero and zero, and that was registered before the run. The decision procedure is engine agnostic by confidence value and needs two numbers, and the specification forbids a self reported confidence from feeding the threshold policy. The only pair satisfying both is zero and zero. The two consequences were written down in advance: the language model gets no threshold protection and accuses on every committed prediction, and the low confidence finding becomes unreachable for it. Deriving a boundary on the self reported scale against a precision target, the way the n-gram engine's was derived, would probably have raised its finding level precision. That it would have flattered the engine is exactly why it had to be refused before the measurement rather than after.

Chapter 6

The audit engine: what gets said out loud

A classifier produces a label. An auditor produces an accusation. The distance between those two is the audit engine, and it is where most of the decisions that a user actually experiences are made.

6.1 The decision procedure

The procedure is ordered, first match wins, and it runs per control:

1. If the control is undetectable, emit `UNDETECTABLE_FIELD` with the reason and stop.
2. If a declaration is present and parses to no token, emit `AUTOCOMPLETE_OFF` and stop.
3. If a declaration is present and is not a specification token, emit `OFF_SPEC_TOKEN` naming the token that was meant, and stop.
4. If a declaration is present and the classifier is confident it names a different field, emit `WRONG_AUTOCOMPLETE` and stop.
5. If no declaration is present and the classifier is confident, emit `MISSING_AUTOCOMPLETE` with the token to add.
6. Otherwise emit the near miss or structural findings that apply, or nothing.

Three places in that ordering had to be read rather than transcribed, and each is a point where two parts of the specification pull against each other.

The composite finding has no branch of its own in the published pseudocode. Its trigger reads as *an inferred composite*, which taken literally fires a warning on every correctly declared month and year input, while the corpus design makes any finding above the lowest severity on a correct form a defect. Both hold only if the finding is about the *absence* of the declaration, so it sits where the missing declaration finding sits and carries the composite's own fix text.

The off switch branch has no stop marker in the pseudocode. Two other branches carry one and this one does not, but a declaration of `off` parses to no token, so without the stop a field collects both that finding and the missing declaration finding: two accusations with two contradictory fixes. The prose above the pseudocode says first match wins, so it stops.

The cross origin frame fix text contradicts the extractor's own guidance. The generic template ends by suggesting the control be exposed in light DOM or the attribute declared on

the host, which is sound for a closed shadow root and nonsense for a frame the page does not own. The more specific rule wins, and the override is data beside the template rather than a branch inside a renderer.

6.2 Thresholds are per engine, and one of them is a mapping

Table 6.1: One threshold block per engine. An engine with no block is an error rather than an inheritance.

Engine	tau high	tau low	Target	Basis
rules	0.7000	0.4000	null	rule-tier-band-mapping
ngram	0.8964	0.6278	0.9800	dev split of the seed 20260825 corpus, calibrated n-gram engine
llm	0.0000	0.0000	null	not a boundary: the self-reported scale decides nothing (spec section 12.1)
bert	0.8813	0.4609	0.9800	dev split of the seed 20260825 corpus, calibrated INT8 encoder. Derived under the identical policy of experiments/predictions/p4-threshold-derivation.md, before the test-split run and before the ship condition of spec section 10.7 was evaluated

Source: `src/autofill_audit/audit/thresholds.json`

Table 6.1 carries a genuine asymmetry and the report should not smooth it.

The n-gram engine’s boundary is derived on the development split against a precision target that was committed before the model existed. The rule engine’s is a documented mapping from its six tiers onto the same two bands, and every key in its block that would hold a measurement is null. That is deliberate. A regular expression table has no probability, and printing a plausible looking number in a file the runtime reads would be a law 3 violation with a straight face. The renderers say in words that a rule tier confidence is a tier and not a frequency.

The language model’s block is zero and zero, for the reason given in Chapter 5: a self reported confidence is forbidden from gating anything, and the engine still needs two numbers.

The fourth block belongs to an engine this build does not offer, and it is here on purpose. The transformer of Section 8.10 was measured against the test split with that boundary in place, and a threshold file that dropped the block afterwards would make a committed result impossible to reproduce from this repository. The block was written before the run and its retention was decided before the verdict, precisely so that keeping it could not look like a decision taken afterwards. A block here is not a claim that the tool offers the engine; the engine list in the classifier ladder is what makes that claim, and the basis string in the block says which of the two states it is in.

Two things about Table 6.2 need saying rather than leaving in a file.

A precision figure is only as strong as its denominator. A precision of one over a handful of accusations and a precision of one over a thousand are the same number and are not the same claim. The derived block therefore records the denominators beside the rates, so a reader does not have to go and find out which it is.

A high precision target makes a much quieter tool. At this boundary the model raises far fewer accusations than there are fields that need one. That is what a pre-registered high precision target buys on a model whose calibrated confidences are honest about how often it is

Table 6.2: The n-gram engine’s derived boundary and what it bought on the development split. The precision target was registered before the model existed and the derivation script was run unmodified.

Derived quantity	Value
Target precision	0.9800
tau high	0.8964
tau low	0.6278
Dev precision at tau high	0.9954
Dev recall at tau high	0.3701
Dev accusations	216
Dev fields needing an accusation	581

Source: `src/autofill_audit/audit/thresholds.json`

right, and it is the correct outcome of the policy rather than a failure of it. The policy exists because a false critical costs a developer far more than a missed one, and this is what *far more* looks like when it is taken literally.

The two engines’ boundaries also had to stay separate. The derived n-gram boundary sits far above the rule engine’s confident band, and a single pair of thresholds across both would have moved every rule engine finding on every page as a side effect of training a model. The threshold document therefore carries a block per engine, the loader takes an engine name, and an engine with no block raises rather than inheriting one. That was designed to bite when the language model arrived without a block, and it did, which is how the zero and zero decision came to be made deliberately and in advance.

6.3 The finding catalogue, in brief

Severity	Codes
critical	MISSING_AUTOCOMPLETE, WRONG_AUTOCOMPLETE, OFF_SPEC_TOKEN
warning	AUTOCOMPLETE_OFF, UNLABELED_FIELD, PLACEHOLDER_AS_LABEL, SPLIT_FIELD, COMPOSITE_FIELD, UNDETECTABLE_FIELD
info	GENERIC_IDENTIFIER, WRONG_INPUT_TYPE, MISSING_NAME_ATTR, EXTRACTION_INCOMPLETE
note	LOW_CONFIDENCE

Table 6.3: The finding catalogue by severity. Each code carries a trigger, a named evidence list, a confidence or a documented tier, and a one line fix.

Each carries a trigger, a severity, a named evidence list, a confidence or a documented tier, and a one line fix as text. There is no mode that edits the user’s markup, because rewriting a production template from a probabilistic classifier’s output is exactly the failure the first law exists to prevent.

6.4 A worked example

The following is real output against a checkout fixture authored in this repository to get things wrong in the ways real checkouts get them wrong. Nothing is edited except for trimming the middle of the table.

```
$ autofill-audit audit tests/fixtures/checkout_hostile.html
autofill-audit file:///.../tests/fixtures/checkout_hostile.html

critical (6)
```

```

control      finding
.....
#ck-email    MISSING_AUTOCOMPLETE
              add autocomplete="email" to #ck-email
              evidence: declaration:absent, label:email-words
              [rule tier HIGH]
#ck-postcode OFF_SPEC_TOKEN
              autocomplete="zipcode" is not a valid autofill
              token; use "postal-code"
              evidence: declaration:off-spec, label:postcode-words
              [rule tier HIGH]
#ck-holder   WRONG_AUTOCOMPLETE
              #ck-holder declares autocomplete="name" but looks
              like cc-name; change to autocomplete="cc-name"
              evidence: declaration:token-mismatch,
              label:cardholder-words [rule tier HIGH]

warning (8)

control      finding
.....
#input7      PLACEHOLDER_AS_LABEL
              #input7 uses a placeholder as its label; add a
              real <label>
              evidence: structure:placeholder-is-the-only-label
frame[#hosted-pan] UNDETECTABLE_FIELD
              the control is inside a cross-origin frame, which
              is how hosted payment fields are built on purpose;
              this is not necessarily a defect. Audit that
              document separately
              evidence: structure:undetectable

$ echo $?
1

```

The before and after that a developer actually applies is one attribute:

```

before, nothing tells the browser what this control is for:

<input type="text" id="ck-email" placeholder="name@example.com">

after, one attribute, and the browser fills it:

<input type="email" id="ck-email" autocomplete="email"
      placeholder="name@example.com">

```

Three properties of that output are the whole design. Every finding names its evidence, so it can be argued with. The rule engine's confidence is a tier and prints as a tier, because turning a regular expression table's output into a percentage would assert a frequency nobody has measured. And a correct field is met with silence: the correctly declared card number input on that page does not appear in the report at all.

6.5 The exit code contract

The process exits non zero when a finding at or above the configured failure threshold is present, which is what lets the tool drop into a pipeline without a wrapper script. The threshold is configurable, the default is the critical severity, and the contract is exercised by end to end tests that run the tool in a subprocess so that the exit codes are the ones a shell sees rather than the ones a test harness believes.

Chapter 7

Method

7.1 What one run is

An evaluation run takes one split, one engine and one seed, loads every form in that split through a real browser, extracts descriptors, classifies every keyed field, runs the audit engine over the same page, and writes five files: a run log with one row per classified field, a manifest, a metrics document, a confusion document, and a findings file.

The run log row carries the form identifier, the template, the locale, the tier, the field selector, the answer key label, the predicted label, the confidence, the kind of confidence, the runner up, the declared token, the per field classification latency, and the engine's own name. The manifest carries the schema version, the commit and whether the tree was dirty, resolved dependency versions, the corpus manifest digest, the split file digest, the seed, the engine's self description, the command line, a machine descriptor and the start and end timestamps.

Three of those deserve a sentence each.

The engine field carries the engine's own name. The specification uses one label in its run log schema and a different one in its headline table for the same engine. Carrying two names for one engine across a repository is how a comparison table becomes ambiguous, so the run log records what the engine calls itself and a report labels its column however it likes.

The confidence kind is mapped rather than copied. The schema's vocabulary and the engines' own words are not the same strings, because the engines' words were made load bearing in the renderers first. The mapping is written down once and is total: an engine whose word is not in it raises rather than defaulting to something plausible.

The findings file is a fifth artefact the specification does not list. Whether a page *needed* an accusation depends on the difference between declaring nothing, declaring off, and declaring a token outside the specification, and the run log row carries a declared token that is null in all three cases. Without the extra file the finding level significance test could not be reproduced from committed artefacts, and a number nobody can recompute is a number law 3 does not allow into a report.

7.2 Metrics

Macro-F1 and micro-F1 over the union of observed labels, per slice; per label precision, recall and F1; a confusion table; abstention rate and accuracy on the fields the engine did commit to; finding level precision and recall per finding code; and per field latency percentiles beside the wall clock breakdown.

Both averages are always reported. They disagree here about which of the two classical engines leads, and a table carrying one of them would be picking a winner by picking a denominator. The mechanism of the disagreement is the abstention asymmetry and it is explained where the numbers are, in Chapter 8.

A cell is reported only when it holds enough fields. Below the reporting minimum a cell renders the words *insufficient data*, not a blank, because a blank reads as an oversight and this is a decision. The threshold is a single constant in the metrics module, imported by everything that reports, with no per table override, and it was chosen before any corpus had been generated and before any accuracy had been computed, which is the only point at which it could be chosen without knowledge of its effect. On this benchmark it fires on exactly one thing, and what it fires on is itself a finding: see Section 8.5.

7.3 The statistical procedure

The unit of resampling is the template, because fields inside one template share an author. The test is a paired sign flip permutation at the cluster level, exhaustive when the arrangement space is small enough to enumerate, with Benjamini and Hochberg step up correction across the whole comparison family [3], and a practical effect gate expressed as an absolute difference in the metric.

Table 7.1: The pre-registered statistical policy and the power the realised design has. Every value in the upper half was fixed before the runs.

Design quantity	Value
Resampling unit	template\{}_id
Clusters in the test split	10
Sign flip arrangements	1024
Smallest attainable two sided p	0.001953
Exhaustive rather than sampled	yes
Alpha	0.050000
False discovery rate	0.050000
Practical threshold, absolute	0.020000
Comparison family size	33
Seed	20260825

Source: `experiments/results/analysis/2026-08-26T21-20-11Z_p6-analysis_6457ac7/analysis.json`

Two decisions in Table 7.1 were fixed while fixing them was still costless, and both would have been contaminated by seeing the numbers.

The comparison family is all three engine pairs over the same eleven comparisons, corrected together. A three engine benchmark could reasonably have corrected across one pair, or two, or all three. The temptation is specific and foreseeable: the comparison this project exists to make is the one against the language model, and a family containing only that pair would be a third the size and would produce smaller adjusted p values for exactly the comparisons the project most wants to be able to report. The size was therefore fixed in the prediction file, along with the stated consequence that every comparison between the two classical engines would carry a larger adjusted p than the same comparison had carried when it was corrected inside a smaller family. It did, and nobody can report that as a change in the underlying result, because the file said it would happen.

The practical effect gate is absolute. The external harness’s gate is a relative percentage; this project registered an absolute difference in the metric. A relative gate turns one pre-registered rule into a different rule in every slice, because it means something different against a large baseline than against a small one.

7.4 The dependency boundary, and what it cost

Permutation testing, multiple comparison correction and the verdict vocabulary are not implemented here. They live in a separate, independently released evaluation harness [2] that this project consumes as an ordinary dependency: not vendored, not submoduled, not copied file by file.

That separation is a deliberate engineering claim. A piece of infrastructure with exactly one consumer has not been shown to be infrastructure; it has been shown to be part of that one program. This project is the second consumer, across a real package boundary, and the friction the boundary exposes is the actual finding. Vendoring the code would have destroyed that evidence by making the boundary unobservable.

The friction split cleanly, and the split is more interesting than either outcome would have been alone.

The statistical primitives fit exactly and were used unchanged. The p value estimator takes the null distribution as an argument, which is precisely the seam a caller with its own resampling scheme needs. The correction procedure needed nothing at all. Its verdict vocabulary already contained the three categories required plus a fourth for a design whose smallest attainable p value sits above alpha, which turned out to be the category every comparison in the first analysis landed in.

Everything shaped like a training run did not fit. The ingestion layer claims a run log of this shape and then refuses it for having no step field, and there is no step to add, because the file has no time axis. No public entry point accepts a cluster assignment. Neither comparison mode is paired, and both reduce a metric series to a final window mean, which a categorical per field outcome does not have.

Five issues are filed on that repository, each with a concrete API proposal, and the ledger in this repository records each one with its status and the workaround, if any, that stands here in the meantime. Two workarounds exist and both are labelled in every result file they produce, because a workaround that is invisible in the output is indistinguishable from a capability: the clustered null is built here and handed to the harness's estimator, and the practical effect gate is applied here because the harness's is relative.

The anticipated friction was aimed one layer too low. The ledger predicted, before the integration began, that categorical outcome support in the permutation machinery would be the problem. It was not needed at all: the estimator never sees an outcome, only an effect and a null, both numbers. The categorical part of the problem lives entirely in the statistic and the statistic is the caller's. What actually broke was one layer above, in the data model.

One consequence of the boundary is stated here rather than discovered later. The harness is a development dependency, not a runtime one, because installing it pulls more than a dozen transitive packages including a training metrics logging stack, and the audit path never imports it. **The shipped tool therefore cannot compute its own significance tests.** The narrower extra that would fix it is proposed in the ledger.

7.5 Reproducibility, and the one artefact that is marked dirty

One seed reaches the generator, the split assignment, the training script and any sampling in evaluation, through explicitly passed generator objects. The lock file is committed and continuous integration installs from it. Every result directory carries a manifest. Model artefacts are content addressed into every run that used them. Nothing is ever regenerated to fix a number: if a result

is wrong, a new run is made with a new identifier and the old one stays, because result files are append only history.

A run produced from a working tree with uncommitted changes is marked dirty in its manifest and is not citable, and the traceability job fails on a citation of one. Every evaluation result cited anywhere in this report is marked clean.

One artefact is not, and it is the training manifest of the shipped model. The model was fitted in the same working session that authored the new corpus templates, so the tree carried uncommitted changes at the moment of fitting, and the manifest says so honestly rather than hiding it. What follows from that is worth being precise about rather than either ignoring or overstating:

- The model artefacts themselves are committed and are content addressed into every evaluation run manifest by digest, so *which* model produced the headline numbers is not in doubt.
- The corpus is content addressed the same way, and its digest in the training manifest matches the digest in every evaluation manifest, so the model was fitted on the same corpus it was evaluated against.
- What is genuinely weaker is the development split evidence: the numbers in Tables 5.1, 5.2, 5.4 and 5.3 come from a run whose tree state was not pinned to a commit. They are reported because they describe how the shipped model was built and they should not be silently dropped, and they are flagged here because a reader is entitled to know that they sit one notch below the test split numbers on the same scale the project applies to everything else.
- The fix is procedural rather than technical, it is not applied retroactively because that would be a regeneration, and it is listed in Chapter 12: commit the corpus change first, then train from the clean tree.

Chapter 8

The headline experiment

8.1 The answer, first

Does a fifty kilobyte linear model that runs in microseconds on a CPU get close enough to a twelve billion parameter model to be the right default for a developer tool?

One correction to the question before answering it. The size in it is the build specification’s own framing, written before the model existed, and the exported graph is larger than that. The artefact is committed under `models/` and anybody can check its size directly. Nothing in the answer turns on the figure: the argument is about a linear model that runs on a CPU in microseconds per field against a twelve billion parameter model on an accelerator, and that contrast is measured rather than asserted.

The premise did not survive the measurement. The linear model did not have to get close, because it beat the language model on both averages. The rule table beat both.

Table 8.1: The headline benchmark: three engines on the test split, both averages, and the fourth row that does not exist.

Engine	Fields	Forms	Templates	Labels	macro-F1	micro-F1
rules	2317	240	10	36	0.7911	0.7678
ngram	2317	240	10	39	0.7394	0.8226
llm	2317	240	10	38	0.6353	0.6953
bert-onnx-int8	absent	absent	absent	absent	absent	absent

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

The language model reached 0.6353 macro-F1 and 0.6953 micro-F1; the n-gram model 0.7394 and 0.8226; the rule table 0.7911 and 0.7678.¹ On this corpus, at this quantization, with this prompt, the small linear model is not the compromise. It is the better classifier, and it is also several thousand times faster, needs no accelerator, and adds nothing meaningful to an install.

The fourth row of Table 8.1 reads *absent* in words rather than being left blank. A blank cell in a comparison table reads as a measurement of zero, which would be a fabricated number in the one place a reader trusts most.

That row is absent for a different reason from the one it originally carried, and the difference is

¹Every number in this paragraph is a generated macro. The values are in Table 8.1 and the paths they came from are printed under it.

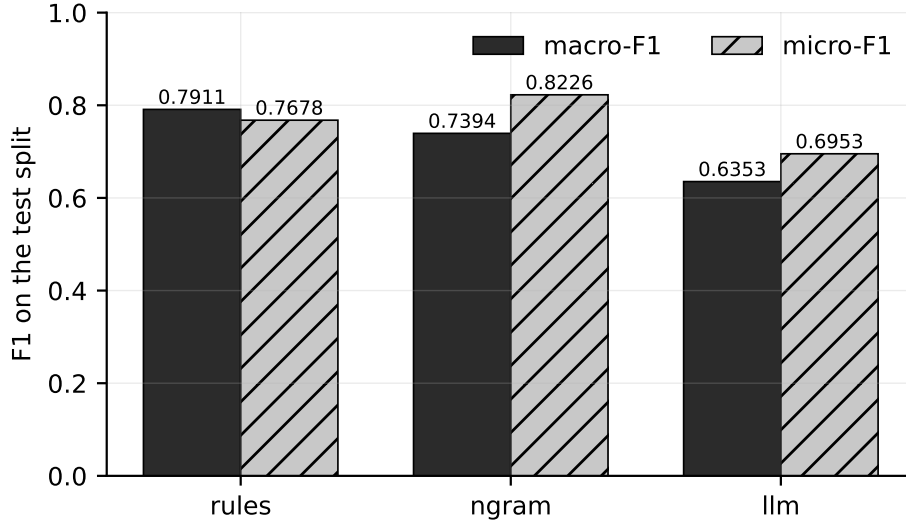


Figure 8.1: Both averages for all three engines. They are shown together because they disagree about which classical engine leads and agree about which came last, and a chart carrying one of them would pick a winner by picking a denominator.

worth a sentence here rather than only later. The transformer rung was specified as an optional later phase, and it was then built, fine tuned, quantized and measured against this same test split. It is the most accurate classifier this project has produced and it did not ship, because it missed a latency budget that was fixed before it was trained. This table is the three engine benchmark it was not part of. Its own run, its result file, and the arithmetic that rejected it are in Section 8.10.

8.2 The two averages disagree, and the disagreement is the mechanism

Macro-F1 says the rule table wins. Micro-F1 says the n-gram model wins. Both are in the same files and both are correct.

The mechanism is abstention. The rule table answers UNKNOWN on 0.2620 of the fields and is right on 0.9702 of what it does answer. The n-gram model abstains on 0.0363 and is right on 0.8204. Abstention costs almost nothing under macro averaging, where an abstention is a miss on one class, and costs a full share of the field weighted denominator under micro averaging. A model that answers nearly everything therefore earns field weighted accuracy and pays label weighted average every time it guesses a rare class wrongly.

Table 8.2: Abstention, and accuracy on the fields each engine did commit to.

Engine	Fields	Answered UNKNOWN	Abstention rate	Accuracy when committed
rules	2317	607	0.2620	0.9702
ngram	2317	84	0.0363	0.8204
llm	2317	59	0.0255	0.7037

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

That is not the rule table being timid. On this corpus abstention is the better policy, and the rule table has it by construction, because its default branch is a tier that never clears a threshold.

8.3 The language model abstained least of all

The system prompt tells the model to answer UNKNOWN when the evidence is insufficient and not to guess. It abstained on 0.0255 of fields, less than either classical engine, and was right on 0.7037 of what it committed to, less than either (Table 8.2).

Instructing a model to decline is not equivalent to giving it a threshold, and this is the measurement of that gap. It is also the prediction that failed in the most useful direction: abstention was registered to land between the two classical engines and landed below both.

8.4 Locale and tier

Table 8.3: Seen locales against the held out locale. The unseen locale column is the multilingual claim’s actual test: French forms whose templates are also unseen.

Engine	Seen macro-F1	Seen micro-F1	Unseen macro-F1	Unseen micro-F1
rules	0.7991	0.7743	0.7276	0.7340
ngram	0.7735	0.8532	0.5637	0.6649
llm	0.6373	0.6981	0.6045	0.6809

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Table 8.4: Macro-F1 by locale.

Locale	Fields	rules	ngram	llm
de-DE	376	0.7259	0.7814	0.5906
en-GB	392	0.7904	0.8128	0.6491
en-NG	377	0.7998	0.8595	0.6643
en-US	392	0.8033	0.8024	0.6480
fr-FR	376	0.7276	0.5637	0.6045
ja-JP	404	0.8285	0.7599	0.6345

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

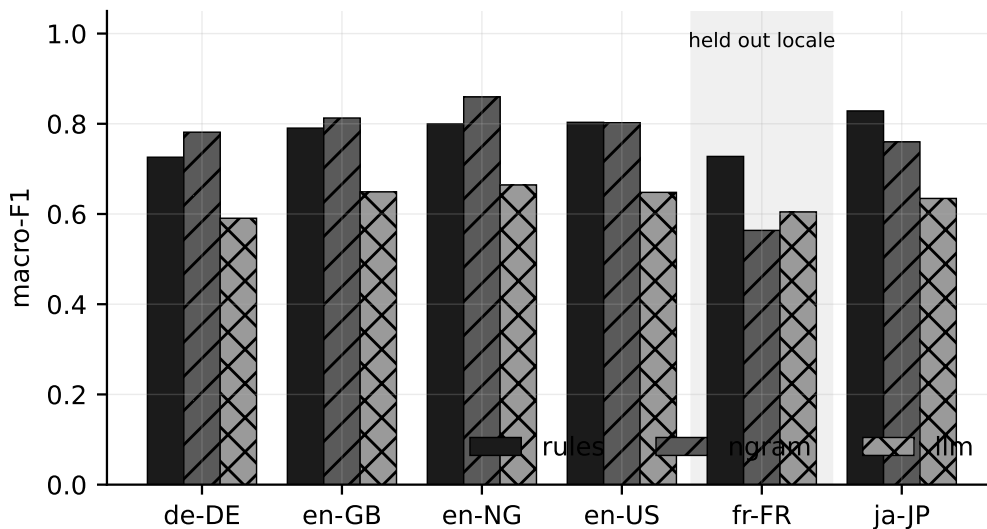


Figure 8.2: Macro-F1 by locale, with the held out locale shaded. It is the only column in which the language model leads the n-gram model, and it is also a column in which the rule table leads them both.

The held out locale is where broad pretrained world knowledge was supposed to pay. It is the one slice where the language model leads the n-gram model, 0.6045 against 0.5637, and Chapter 9

shows that this difference does not survive correction across the family. It is also a slice where the rule table leads both, at 0.7276. A vocabulary of localised regular expressions generalises to an unseen locale better than either learned system does here, which is an uncomfortable result and is printed as it stands.

Table 8.5: Macro-F1 by markup quality tier.

Markup tier	Fields	rules	ngram	llm
clean	520	0.8677	0.9350	0.6696
partial	519	0.8677	0.9290	0.6522
mixed	639	0.8005	0.7151	0.6352
hostile	639	0.4681	0.5409	0.5265

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

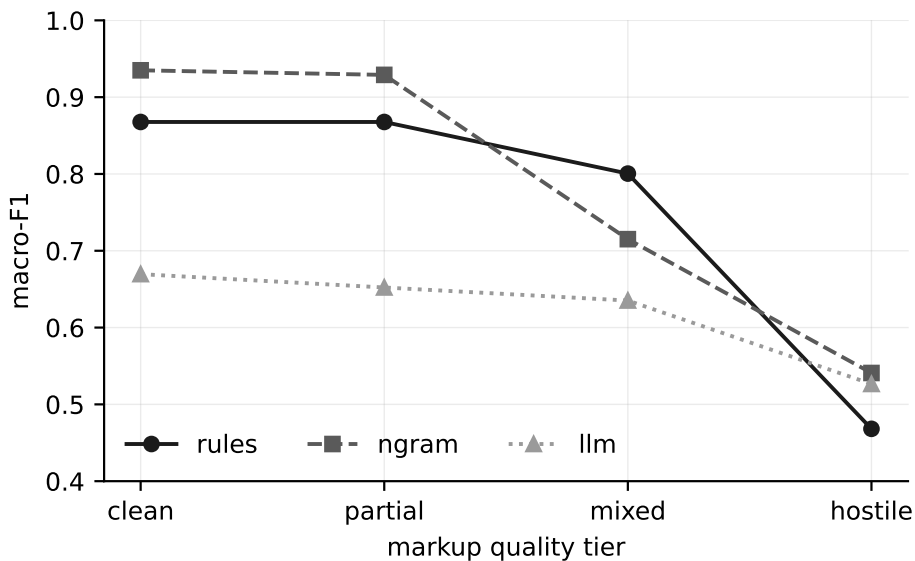


Figure 8.3: The tier profile, which is the most informative single picture in the benchmark. The two classical engines have strong and opposite slopes; the language model is nearly flat and below both until the tiers converge.

Figure 8.3 is worth reading slowly.

The n-gram model is the best engine on clean and partial markup and the worst classical engine on mixed markup. It leads on clean at 0.9350 against 0.8677, and trails on mixed at 0.7151 against 0.8005. The mixed tier is the one built to look like a real page whose sections were written by different teams at different times. A model that is better on each tier separately and worse on the tier that mixes them is worth a paragraph rather than a footnote: it suggests the model is picking up a per page consistency that the mixed tier deliberately breaks.

The hostile tier is where every engine is worst and where the three converge. The rule table falls furthest, to 0.4681, because stripping labels removes most of what a vocabulary of regular expressions has to work with. The n-gram model reaches 0.5409 and the language model 0.5265. This is the slice where broad pretrained knowledge was predicted to help most, and none of the differences on it is significant.

The full locale by tier grid, all twenty four cells for all three engines, is Table A.4 in Appendix A. Every cell clears the reporting minimum, so nothing in it is suppressed.

8.5 What a developer actually experiences

The finding level numbers matter more than the classifier numbers, because they are what a user of the tool sees. They run through the decision procedure and the thresholds rather than reporting raw classifier accuracy.

Table 8.6: Finding level precision and recall. Precision is withheld for the two classical engines on the second code because neither made enough accusations to clear the reporting minimum, and that asymmetry is the finding rather than a formatting inconsistency.

Code	Engine	Accusations	Correct	Needed	Precision	Recall
MISSING_AUTOCOMPLETE	rules	449	449	790	1.0000	0.5684
	ngram	334	325	790	0.9731	0.4114
	llm	692	540	790	0.7803	0.6835
WRONG_AUTOCOMPLETE	rules	28	28	30	insufficient data	0.9333
	ngram	23	21	30	insufficient data	0.7000
	llm	125	29	30	0.2320	0.9667

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

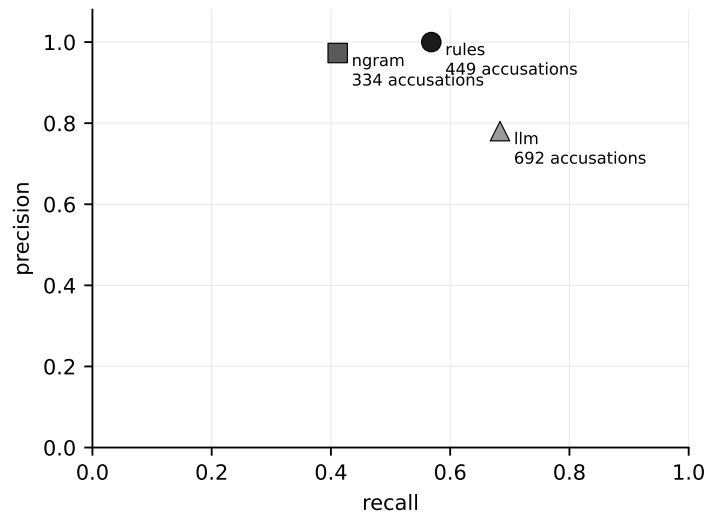


Figure 8.4: Where each engine lands on precision against recall for the missing declaration finding. **This compares three threshold policies, not three classifiers.** Each engine speaks under its own rule about when to speak, and the spread along the horizontal axis is mostly a picture of those rules.

The rule table was right about every field it accused. It raised 449 missing declaration accusations and 0 of them were wrong, at a precision of 1.0000 and a recall of 0.5684 against 790 fields that needed one.

The n-gram model made this project's first wrong accusations. It raised 334 and was wrong about 9 of them. That is the first time in this project that an engine told a developer to add a token the answer key disagrees with, and it deserves recording rather than rounding. The threshold policy did not change and the derivation script was run unmodified; what changed is that the derived boundary now sits on a model whose development precision at that point was itself below one, so a test split precision below one is the design working rather than failing. A precision target is a target, not a guarantee, and a project that reported the target as though it were the outcome would be reporting a policy as a measurement.

The language model found the most real defects and was wrong about roughly one accusation in five. It raised 692 accusations, 152 of them wrong, at a precision of 0.7803 and

the highest recall of the three at 0.6835. On a developer tool that means a fifth of the work it hands you is work you should not do.

On the second accusation code the asymmetry is starker. 30 fields needed it, which clears the reporting minimum of 30, so every recall is reported. Precision is withheld for both classical engines because neither made enough accusations to be measured, and is reported for the language model at 0.2320 because it made 125. The engine whose precision can be reported is the one that accused often enough to be measured, and that is the finding.

None of this is a property of the language model as a classifier. It is a property of its threshold block, which is zero and zero because a self reported confidence is forbidden from gating anything, and which was registered before the run. **A finding level comparison between these engines is a comparison of three policies, and that sentence has to travel with the table.**

8.6 Latency, and the context that decides whether it matters

Table 8.7: Per field latency in microseconds, and the share of each run’s wall clock spent loading and extracting the page.

Engine	p50	p95	p99	Load and extract share
rules	35.2	134.0	194.7	0.9912
ngram	165.6	261.5	355.1	0.9874
llm	524189.3	674727.4	774080.6	0.0537

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Table 8.8: Where each run’s wall clock actually went, in seconds.

Engine	Load	Extract	Classify	Render	Total
rules	131.7	4.4	0.1	0.01	137.4
ngram	131.8	4.4	0.5	0.02	137.9
llm	133.0	4.9	1280.2	0.03	2565.8

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

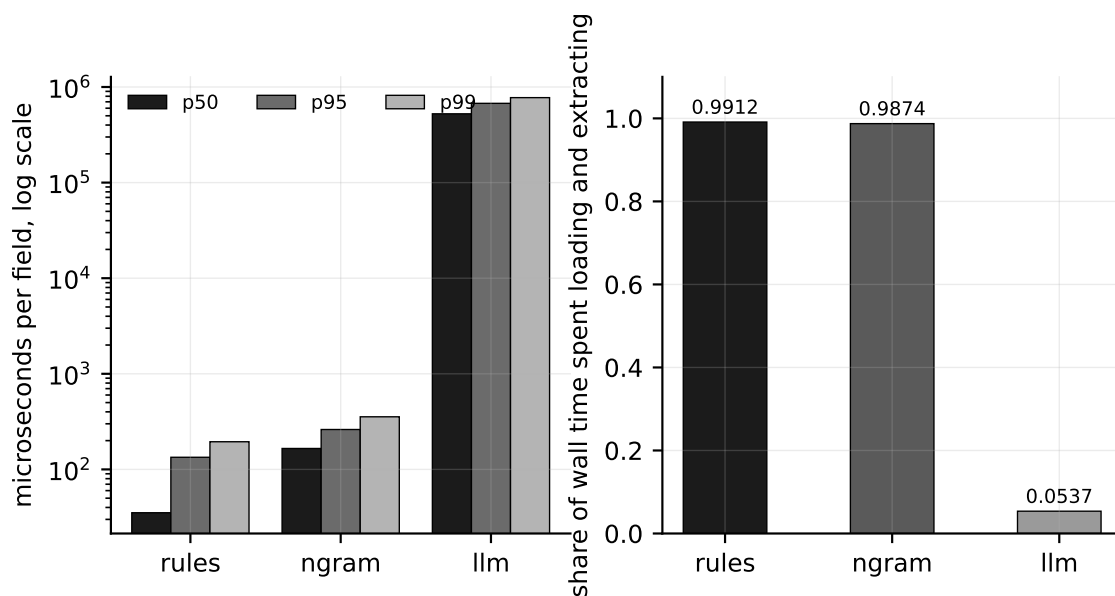


Figure 8.5: Latency percentiles on a log axis, beside the share of wall time the browser takes. The log axis is not a stylistic choice: a linear one is unreadable across this range.

The two classical engines run on a CPU with the inference session warmed and its threads pinned, and both are comfortably inside a millisecond per field: 134.0 and 261.5 microseconds at the ninety fifth percentile. The language model runs on a warmed local accelerator and reaches 674727.4, about 5037 times the rule table’s figure at the same percentile.

The language model’s figure is a share, not a per call measurement. It classifies a whole page in one request, so its per field number is a batch’s elapsed time divided by the fields in that batch. Batching amortises the round trip, which makes the figure flattering rather than harsh, and it is not comparable against a per call measurement. The engine’s own description says so and the run manifest records it.

The row that turns a latency difference into a product difference is the share of wall time. For the rule engine the browser is 0.9912 of the run and for the n-gram engine 0.9874, so the classifier is a rounding error on a page load and celebrating the difference between their latency columns would be celebrating nothing. For the language model the browser is 0.0537 and the classifier is the rest. One is a classifier you do not wait for. The other is one you do.

In wall clock, over the identical page set: about 2.3 minutes for the rule engine, 2.3 for the n-gram engine, and 42.8 for the language model.

This is the finding that survives contact with real pages. The accuracy gap depends on the corpus being synthetic. The runtime inversion does not.

8.7 What the language model run did, and what it did not

Table 8.9: The language model run, including the failure mode that did not occur.

Quantity	Value
Requests	480
Attempts	480
Requests needing a retry	0
Requests still failing after the retry	0
Fields scored UNKNOWN by a schema failure	0
Smallest batch	3
Largest batch	23
Median batch	8
Fields per request cap	40
Prompt tokens	677632
Completion tokens	159126
Monetary cost	null

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Schema compliance was perfect, and that was not predicted. Across 480 requests, 0 needed a retry, 0 still failed after one, and 0 fields were scored UNKNOWN by a schema failure. Passing the response schema with the request and letting the server constrain decoding appears to remove the failure mode outright. The retry ladder is still there and is still tested against recorded transcripts including deliberately malformed ones; on this run it never fired. A benchmark whose most interesting failure mode did not occur is a result, and the honest report of it is the zero rather than a paragraph about robustness.

Cost is recorded as null rather than zero. The calls were local. Zero is a measurement and the electricity was not free; null is the absence of one. No price list was retrieved, so no list price estimate is computed, and the price table document in this repository deliberately contains no prices.

8.8 The engine order in the benchmark was a methodological choice

The engines ran with the language model first, which is not the order the tables present. The two classical engines take a couple of minutes each of browser time with no inference call in them, so running the language model last would have meant several idle minutes before its first call, against a bounded keep alive policy. A keep alive expiry inside a measured run would have put a cold model load into the per field latency distribution. Warming immediately before and running it first avoids measuring the disk.

8.9 The two classical engines reproduced their previous run exactly

Every prediction and every finding, field for field across all 2317 fields, and every metric identical to the previous measurement on the same corpus. Only the latency columns differ, because those are re-measured rather than recomputed.

That was not the purpose of re-running them. It is the strongest reproducibility evidence this project has produced and it is worth the sentence.

8.10 The transformer that led the table and did not ship

A distilled multilingual encoder was fine tuned on the same training split, exported to ONNX, quantized to eight bit integers with dynamic activations, and measured against this same test split through the same runner [12]. It scores 0.8026 macro-F1 and 0.8895 micro-F1, which makes it the most accurate classifier this project has measured, and it is not an engine the tool offers.

The condition that rejected it was fixed before it existed, in the first commit of the phase, ahead of the training dependencies being installed. It is a conjunction of three bars and the verdict is arithmetic rather than editorial. The unseen locale had to beat the n-gram model's 0.5637 by at least 0.05, that is reach 0.6137, and it reached 0.7192. The whole split had to stay within 0.01 of the n-gram model's 0.7394, that is not fall below 0.7294, and it reached 0.8026. The ninety fifth percentile latency per field had to sit at or under 5000.0 microseconds, and it measured 14303.1.

Two of three is a no ship, and the condition says so in as many words specifically because two of three with a wide margin on the two is the case where it would be tempting to reconsider. The margins are wide. The engine beat the n gram model on every locale and on every tier: the held out locale by 0.1555, the hostile tier by 0.1123, the whole split by 0.0632, and the recall of MISSING_AUTOCOMPLETE by 0.2709 at a precision of 0.9782 against the n-gram model's 0.9731. Corrected across a family of 66 comparisons over 6 engine pairs, the held out locale, the hostile tier and the recall gain survive and the whole split difference does not.

It still does not ship, because the third bar is a budget rather than a target, and a budget that bends when the numbers are good was never a budget. Per field the engine costs about 55 times the n-gram model at the same percentile on the same pinned single thread session. In whole page terms that turns classification from a fraction of a percent of the run into 0.1476 of it, with the browser still 0.7003. So the cost is not that the tool would feel slow. It is that the classifier's share of the work would grow by a factor of forty in a tool whose own measurements say the classifier was never the bottleneck, while adding a tokenizer, a second model format, and a model file of 136 megabytes that this repository cannot hold.

The parity contract was written in two parts before either export existed, because demanding exactness from a quantizer is demanding that it not work. The first part passed: the FP32 graph

agreed with the trained model on the argmax of every one of the 1596 development rows, with a maximum absolute logit difference of 0.000015 against a ceiling of 0.001. The second part did not. Dynamic INT8 was required to agree with the FP32 graph on at least 0.99 of development rows and to give up at most 0.01 of development macro-F1; it agreed on 0.9787 and gave up 0.0139. Both bounds are missed, neither by much, and both are recorded rather than adjusted. Under the pre-registered rule that is a second and independent failure of the latency bar, which is defined on the quantized engine.

What the specification predicted about this rung was that the advantage would be concentrated in the held out locale and the hostile tier and would be small or negative on the clean tier, where the n-gram model already has the label text. The first two held: both gains are larger than the whole split gain, which is what concentration has to mean to be checkable. The third did not. The clean tier gained 0.0279, where the phase had predicted less than 0.02. It is a small contradiction and it points somewhere: on a clean page the encoder is not seeing anything the linear model cannot see, so the gap that remains is about how the two generalise from a label they have both been shown.

The result that survives this corpus being synthetic is not the accuracy number. It is the shape of the trade. A pretrained multilingual encoder buys most of its advantage exactly where a bag of character n-grams is weakest, on a locale it was never trained on and on markup stripped of its labels, and it charges two orders of magnitude of arithmetic for it.

Chapter 9

Is the difference significant

9.1 What was run

A paired sign flip permutation clustered at the template level, exhaustive over the full arrangement space, with step up correction across a family of 33 comparisons, and an absolute practical effect gate. The design quantities are in Table 7.1.

The test split holds 10 templates, which gives 1024 arrangements and a smallest attainable two sided p value of 0.001953, comfortably below the pre-registered level of 0.05. **Every comparison in this analysis is decidable**, which was not true of the first measurement this project made and is the whole point of the corpus repair described in Chapter 3.

9.2 The outcome categories, including the empty ones

Table 9.1: Every verdict category in the family of comparisons, including the two that came back empty. A summary listing only the categories that occurred would drop them on exactly the runs where nothing landed in them.

Verdict	Comparisons
improvement	11
regression	5
significant but below the practical threshold	0
no significant change	17
inconclusive: the design cannot reach alpha	0
total	33

Source: `experiments/results/analysis/2026-08-26T21-20-11Z_p6-analysis_6457ac7/analysis.json`

Table 9.1 carries all five categories. Two are worth pointing at because they are empty.

Significant but below the practical threshold: 0. This category exists because a difference can be certain and still be too small to act on, and a reporting scheme that silently merges it into either neighbour is a reporting scheme that will one day sell a rounding error as a result. On this family nothing landed in it. It is printed anyway.

Inconclusive because the design cannot reach alpha: 0. This is the category that held all 11 comparisons of the first analysis, when the test split had 5 clusters and a smallest attainable p of 0.0625. It is now empty, which is the measurable result of the repair.

Table 9.2: All comparisons in the family, read in the direction they were measured. The full table is Table A.2 in Appendix A; this one repeats its shape for reference.

Cross check	Comparisons
Comparisons checked	33
Verdicts that agree	33
Verdicts that disagree	0

Source: `experiments/results/analysis/2026-08-26T21-20-11Z_p6-analysis_6457ac7/analysis.json`

9.3 What is certified, and what is not

Certified against the language model, after correction: the rule table leads it overall and on the seen locales, and both classical engines lead it on clean and on partial markup. The language model’s missing declaration recall is certified better than both, and its precision on both accusation codes is certified worse than both.

Not certified: everything on the hostile tier, everything on the held out locale, and the language model against the n-gram model overall. Those are precisely the slices where the language model was expected to do well, and the honest report of them is that the differences are not large enough to call at 10 clusters, in either direction.

Not the same statement as no difference. Comparisons in the *no significant change* category are comparisons where this design, with this cluster count, could not resolve the effect. Reading them as evidence of equivalence would be reading the absence of a certificate as a certificate.

The complete list, every comparison with its effect, its exact p value, its adjusted p value and its verdict, is Table A.2.

9.4 One reading hazard, stated explicitly

The comparison names in the result file read `baseline_vs_candidate` and the effect is computed as candidate minus baseline. A comparison named for the language model against the rule table with a verdict of *improvement* therefore means **the rule table improved over the language model**, not the reverse.

The file is self describing: every comparison carries its baseline and its candidate by name. But the string reads backwards at a glance, which is exactly the kind of thing that ends up in a report as a sentence pointing the wrong way. The table generator never renders the comparison name as a sentence. It reads the baseline, the candidate and the effect and builds the sentence, and the *Comparison* column of Table A.2 is that sentence.

9.5 The family size was fixed in advance, and it cost something

Every comparison between the two classical engines carries a larger adjusted p value in this analysis than the same comparison carried when it was corrected inside the smaller family of the previous phase. The underlying p values did not move. The earlier analysis file is untouched and is still in the repository.

That was registered before the runs, with the reason: the comparison this phase existed to make was the one against the language model, and a family containing only that pair would have been a third the size and would have produced smaller adjusted p values for exactly the comparisons the project most wanted to be able to report. Fixing the family size while it was still costless is the only way that sentence can be written afterwards without it being an excuse.

9.6 The harness’s own verdicts, as a cross check

The external harness carries its own end to end verdict machinery. Its statistical gate and its underpowered gate are the ones used above. Its practical gate reads a relative percentage where this project pre-registered an absolute difference, so a disagreement between the two verdict columns would be that gap and nothing else.

It was run alongside on the same numbers as a visible cross check. 33 comparisons were checked and 0 disagreed. That gap therefore changed no verdict on this data, which is worth saying plainly and is not a reason to withdraw the filed issue: a gap that happens not to bite on one dataset is still a gap, and the dataset it would bite on is any slice with a small baseline.

9.7 The secondary analysis, and why nothing cites it

A secondary analysis clustered by form rather than by template reaches significance easily. It is computed, it is in the result file, and it is labelled anticonservative everywhere it appears, because clustering by form asserts that two locales of one template are independent, which is a stronger claim than this project’s design makes.

Nothing in this report, in the README, or in any document in this repository cites it as evidence. It is present so that the difference between the two clusterings is visible rather than assertable, and that is the only reason it is present.

Chapter 10

What was predicted, and what was wrong

10.1 Why the predictions are here at all

A prediction committed after the measurement is not a prediction. Every prediction in this project was written into a file, committed, and a continuous integration job verifies that the prediction commit is an ancestor of the commit carrying the result files it predicts about. That check is the only mechanism that makes the fourth law more than an intention.

The predictions are printed beside the outcomes because that is the only form in which a prediction is worth anything. A report that showed only the outcomes would be indistinguishable from a report whose author had decided afterwards which results were interesting.

10.2 Four held and four did not

The four that failed are the more interesting half, and two of them should be read first.

Schema compliance was predicted to fail sometimes, and never failed. The registered claim was that the retry rate would be above zero and below one request in twenty. It was exactly zero across every request of the run. The mechanism appears to be that passing the response schema with the request lets the server constrain decoding, which removes the failure mode rather than reducing it. The retry ladder was built for a problem that, on this serving route, does not arise. It is still there and still tested, because the next serving route may not have that property.

Abstention went the wrong way. It was registered to land between the two classical engines, on the reasoning that a model instructed to decline would decline more than a model with no such instruction and less than a rule table whose default branch abstains by construction. It landed below both. Instructing a model to decline is not equivalent to giving it a threshold, and this is the measurement of that difference.

The hostile tier prediction was wrong in the direction that matters most. The hostile tier is where labels are stripped and only obfuscated identifiers survive, which is where broad pretrained knowledge was supposed to pay for itself. The language model lost there too.

The confusion prediction was wrong in direction as well as in detail. The failure was expected to be over claiming: fields that are not personal data read as though they were. The largest single confusion is the reverse, fields whose answer key is UNKNOWN read as the label for a control with no autofill meaning. The model declines more readily than predicted on the

Table 10.1: The eight claims registered before the benchmark ran, the number that settled each one, and the outcome. The measured column is read from the result files by the table generator, so a prediction cannot be scored generously by whoever writes the report.

Registered before the run	Measured	Outcome
The language model beats the n-gram model on the hostile tier	llm 0.5265 against ngram 0.5409	wrong: it lost
The language model beats the n-gram model on the held out locale	llm 0.6045 against ngram 0.5637	held, and not certified after correction
The language model does not beat the rule table on the held out locale	llm 0.6045 against rules 0.7276	held
Latency differs by three orders of magnitude or more	p95 674727.4 against 134.0	held
Schema failures above zero and below one request in twenty	0 retried of 480 requests	wrong: exactly none
The n-gram model stays the right default	ngram macro-F1 0.7394 against llm 0.6353	held, for a stronger reason than the one registered
tel-national read as tel occurs; username read as email does not	tel-national as tel 14; user-name as email 0	held in both directions
Abstention lands between the two classical engines	llm 0.0255, ngram 0.0363, rules 0.2620	wrong: below both

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Source: `experiments/results/test/2026-08-26T20-32-48Z_p6-llm_6457ac7/metrics.json`

fields where declining is wrong, while declining less readily overall than either classical engine. Table A.8 in Appendix A gives the counts for every confusion pair the specification named in advance, and Table A.7 gives the largest confusions each engine actually makes.

10.3 The pattern across every phase of this project

Read across all four prediction files rather than only this one, a pattern is visible and it is not flattering to the intuition that produced the predictions.

Every prediction that failed was about a model. Every prediction that held was about the system. The predictions that held were the ones where the mechanism was already understood: page load dominates because a browser is slow; the n-gram engine is slower per field than the rule table because it enumerates n-grams in interpreted code; latency differs by orders of magnitude because one engine is a regular expression match and the other is a forward pass. The ones that failed were the ones where a mechanism was assumed rather than measured: the model will generalise better across locales; broad knowledge will help on degraded markup; instructing a model to abstain will make it abstain.

Two earlier phases are worth listing beside this one, because the pattern is what makes them worth listing.

At the phase that built the n-gram model, the prediction that the derived decision threshold would sit *below* the rule engine’s confident band was wrong in the other direction: it sits well above it. That single wrong prediction is the reason the two engines carry separate threshold blocks, and it is the most consequential wrong prediction in the project.

At the first evaluation phase, three predictions about the model winning all failed, and a fourth about which confusion pairs would appear failed completely: not one of the four pairs named in advance occurred even once, in either direction, for either engine.

10.4 The temptation that was registered against

The prediction file for this benchmark records, in advance, the specific temptation that a three engine comparison creates: to correct across the one engine pair the project cared about rather than across all three. It also records the consequence of refusing that, which is that every comparison between the two classical engines would come out with a larger adjusted p than before.

That is the shape a useful pre-registration takes. It is not a forecast of the result. It is a written commitment to the analysis choices that would otherwise be made after seeing it.

Chapter 11

Limitations

The answer in Chapter 8 is narrower than it sounds, and the limits are part of the answer rather than a disclaimer appended to it. They are in the same document as the result, and the most important ones are repeated in the same paragraph as the result where they first appear.

11.1 What the accuracy result does not license

It does not license any claim that language models are bad at this task. What was measured is one twelve billion parameter model at four bit quantization, with one prompt, no fine tuning, on a synthetic corpus, where the entire input is a short list of attribute strings.

That input shape is close to the worst case for a system whose advantage is world knowledge, and close to the best case for one that memorises naming conventions. A language model asked to reason about a page would have the page. This one had a pruned descriptor, because giving it the raw document while the n-gram model got a feature vector would have measured something else entirely.

The two classical engines were fitted on this corpus's own training split. The language model met the corpus for the first time at inference. That is a real asymmetry and it points in the direction of the observed result, not against it. A fine tuned model, a better prompt, a larger model, a higher precision quantization, or real pages could each move the result, and none of them was varied.

One prompt was tested. There is no prompt sweep, and there deliberately is not one, because a prompt sweep chosen after seeing the first result is a search for a number. Testing several prompts is future work and it requires a pre-registration of its own.

11.2 The corpus is synthetic, and that is the load bearing limitation

Behaviour on real world pages is unmeasured. Every number in this report is a measurement on forms this repository generated.

The corpus was designed to be adversarial in the places that matter, and the markup quality tiers exist precisely so that the easy case does not dominate the average. But a generated corpus has a generator's habits, and a classifier fitted on it learns those habits along with the phenomenon. The rule table has the same exposure in a different form: its vocabulary was written by the same author as the locale profiles that produce the label text it matches.

That shared authorship is the strongest single reason to distrust the accuracy gap between the classical engines and the language model, and it is why the runtime finding is presented as the more robust one.

11.3 Six locales is not multilingual

Six locales, one of them held out. Every locale in the corpus uses either Latin or Japanese script. There is no right to left locale, no Indic script, no Cyrillic locale, and no locale whose address grammar differs from the ones present in ways the generator does not model.

The normalisation layer was hardened against the whole basic multilingual plane after two property test failures outside the corpus’s own alphabet, so the tokeniser is not the weak point. The locale profiles are, and they are hand written per locale, which means adding a locale is a real piece of work rather than a configuration change.

11.4 What the tool cannot see

Repeated here from Chapter 4 because a reader who skipped to the results should still meet them:

- Closed shadow roots are invisible to any script on the page, including this one.
- Cross origin frames are detected but not read. A hosted payment field is a common and correct pattern, and the finding says so rather than implying a defect.
- Canvas rendered form like widgets are reported as undetectable rather than guessed at.
- Only the step of a multi step form that is currently in the document is audited. There is no click through, because clicking is filling.
- **The tool does not verify that a browser actually autofills.** It verifies that the markup gives the browser what it needs. Those are different claims and only the second one is made.

11.5 The evaluation’s own limitations

The design resolves large effects, not small ones. 10 clusters give 1024 arrangements. That is enough to certify the differences reported as certified in Chapter 9 and not enough to resolve the ones reported as not certified. More templates per family is the only thing that helps; more resamples do not, because the test is already exhaustive.

Two statistical components run on this side of the package boundary. The clustered null and the absolute practical gate are computed here rather than in the harness, because the harness has no public entry point for either. Both are labelled in every result file they produce. The alternative, degrading silently to unclustered resampling, would have made every comparison look more significant than it is, and it was the one outcome ruled out before the phase began.

The evaluation runner classifies every form twice. This is a real defect and it is documented here rather than in an issue nobody reads. The runner classifies once directly, which is where the run log’s predicted label and per field latency come from, and once through the audit engine, which is where the finding codes come from. For the two deterministic engines the passes agree by construction and cost microseconds. For the language model it doubled the inference: the run made twice as many requests as there are forms, and about half its wall clock is the second pass.

Two consequences follow, and the second matters to a reader of Table 8.6:

- The language model benchmark cost about twice what it needed to.
- **A row’s predicted label and its finding codes come from two different draws of a model that is not guaranteed to be deterministic.** Measured on this run, using the deterministic rule engine as a control for the check’s own edge cases, the excess disagreement between the two passes is on the order of two fields in seven hundred. Small, real, and it should be known before anyone builds on the finding level numbers.

It is not fixed in this report’s phase, and the reason is a rule rather than a preference: fixing it changes what the evaluation runner does for every engine, and doing that after seeing the results is the regeneration that this project’s reproducibility rules forbid. The fix is in Chapter 12 and it is to pass the first pass’s predictions into the audit rather than letting the audit reclassify, which also turns the two numbers into one fact.

11.6 Development split evidence is one notch weaker

Stated in full in Chapter 7 and repeated here so that it is met twice: the training manifest of the shipped model is marked as produced from a working tree with uncommitted changes. The model artefacts and the corpus are content addressed into every evaluation manifest, so the identity of what was measured is not in doubt, but the development split tables in Chapter 5 rest on a run whose tree state was not pinned to a commit. They are reported and they are flagged.

11.7 Class imbalance and small denominators

Several labels in the taxonomy are carried by one template in one position, so their per label rows in Table A.6 rest on small denominators. Macro-F1 weights every label equally, which means those rows have the same weight in the headline average as the labels that appear on every form.

That is a deliberate choice rather than an oversight: a label weighted average is the one that notices when an engine quietly stops predicting a rare class. It does mean the macro average is noisier than the micro average, and it is another reason both are reported.

11.8 Cost was not measured because nothing was billed

The monetary cost column is null for every engine. The two classical engines have no provider and the language model ran locally. Zero is a measurement and the electricity was not free; null is the absence of one. No vendor price page was retrieved during this work, so no list price estimate is computed anywhere, and the price table document in this repository contains no prices on purpose. If a cost column with a number in it is wanted, that is a retrieval with a date, not a recollection.

Chapter 12

Future work

Ordered by what would change a conclusion in this report, rather than by what would be pleasant to build.

12.1 Measure something real

The single most valuable next measurement is a small, hand labelled set of real pages. Every accuracy number in this report is a measurement on a synthetic corpus, and the strongest objection to all of them is that a generated corpus has a generator’s habits.

This does not require scraping, which is out of scope and stays out of scope. It requires a small set of pages, labelled by hand against the token set, with the labelling protocol and the disagreement rate published alongside. A hundred real forms with a published inter annotator agreement would tell a reader more about whether the tool works than another thousand generated ones.

The prediction that goes with it should be registered first, and the honest registration is that the rule table’s advantage will shrink, because a substantial part of it is that its vocabulary and the corpus’s label text were written by the same person.

12.2 Fix the double classification in the evaluation runner

Described in Chapter 11. The runner classifies each form twice and the two passes are separate draws for a non deterministic engine, which puts an excess disagreement on the order of two fields in seven hundred between a row’s predicted label and its finding codes.

The fix is to pass the first pass’s predictions into the audit rather than letting the audit reclassify. It is small, it makes the two numbers one fact, and it halves the cost of any future language model benchmark. It was not done in the phase that found it because changing what the runner does for every engine after seeing the results is exactly the regeneration this project’s rules forbid, and because the honest place for it is a phase that re-registers its predictions and re-runs everything.

12.3 Vary the one thing that was held fixed

The language model result rests on one model, one quantization and one prompt. Three variations would each answer a different question, and each needs its own pre-registration:

- **A higher precision quantization of the same model.** Isolates the quantization from the model. Cheap to run and answers an objection that will otherwise be raised forever.

- **A second prompt, registered before it is run.** A prompt sweep chosen after seeing a result is a search for a number; a second prompt registered in advance, with a stated hypothesis about which slice it should move, is a measurement.
- **A fine tuned small model.** The comparison as it stands is between two engines fitted on the corpus and one that met it at inference. Removing that asymmetry is the fairest version of the experiment and it is also the most expensive.

12.4 A smaller encoder than the one that was measured

The transformer rung is no longer future work. It was built, fine tuned, quantized and measured, and Section 8.10 has its numbers: it cleared both accuracy bars of its pre-registered condition, missed the latency budget by a factor of nearly three, and therefore did not ship.

What that leaves open is not whether a transformer helps on this problem, which is now measured, but how much of the help survives shrinking. The engine that was measured is a six layer encoder at a hidden width of seven hundred and sixty eight, and it spends about 55 times the n-gram model's arithmetic per field to buy 0.1555 on the held out locale and 0.1123 on the hostile tier. A model at roughly a quarter of that arithmetic would sit inside the budget if it kept its accuracy.

The prediction worth registering before that measurement is that it would not keep all of it, and the reason to take the measurement anyway is that the accuracy it would have to keep is now a committed number rather than a guess. The two slices to judge it on are the same two: the held out locale, where a multilingual encoder's pretraining is the whole argument for using one, and the hostile tier, where labels have been stripped and a bag of character n-grams has least to work with.

12.5 More templates per family

The design resolves large effects and not small ones, and the only thing that improves that is more templates in the test partition. The arithmetic is in Table 3.2: the relationship between templates and the smallest attainable p value is steep at the low end and flattens quickly.

The cost is the one already paid once: a regeneration, a retraining, a threshold rederivation and a re-measurement, with every recorded number moving at once and an explanation owed in advance.

12.6 Close the cross repository tasks

Five issues are open on the evaluation harness, each with a concrete API proposal, and two of them block things this project would like to state plainly:

- **Publication of the harness to a package index.** Until then the dependency is a source reference rather than a released version, and the project's own rule forbids tagging a stable release while that is true. This is the one open task that is a hard blocker rather than an improvement.
- **A narrower dependency set for the harness.** It currently pulls more than a dozen transitive packages including a training metrics logging stack, which is why it sits in the development extra and why the shipped tool cannot compute its own significance tests.
- Clustered and paired resampling as a public entry point, an absolute practical effect threshold, a stable join key on the verdict output, and a typing marker so that values crossing the boundary are checked.

12.7 Smaller things worth doing

- **Make the Japanese postal mark reachable.** It is in the rule table with a comment saying it cannot fire, because normalisation treats a lone symbol as a delimiter. Making it reachable means changing what counts as a token character, which churns every golden snapshot and every committed fixture expectation, so it is a deliberate change with a cost rather than a quiet fix.
- **Ship a model with the wheel, or a documented way to fetch one.** A published wheel currently carries no model, so an install runs the rule baseline until the user trains one or points the tool at a bundle. The behaviour is documented and prints a line rather than failing silently, but it means the default install is not the default engine the measurements recommend.
- **Widen the hyperparameter grid, with a fresh registration.** The sweep chose the edge of its own grid, so the shipped model may be less regularised than a wider grid would have chosen. Widening it now would be searching for a number; widening it inside a phase that registers first is a measurement.
- **Report the reliability curve per class rather than pooled.** Calibration made the pooled expected calibration error worse while improving macro-F1, and a per class view would say which classes moved and in which direction.

12.8 What is deliberately not future work

Named so that they stay named and stay out.

Crawling. Form filling. Markup rewriting. Scraping training data. A browser extension that duplicates the extractor in a second language and doubles the surface that has to be kept in agreement. Each of those is a boundary rather than a missing feature, and each is stated as a boundary in the project's own documentation.

A form filling agent is a genuinely interesting successor and it is named here so that it stays a separate project rather than becoming a flag on this one.

Appendix A

The long tables

The tables in this appendix are the complete versions of grids the chapters quote from. Every one is generated by `scripts/make_report_tables.py` from a committed result file, and the paths are printed under each.

A.1 Every comparison in the family

Comparison	Metric	Slice	Effect	p	adjusted p	Verdict
rules over llm	macro_f1	all	0.1558	0.00781	0.02578	improvement
rules over llm	macro_f1	seen_locales	0.1618	0.00586	0.02578	improvement
rules over llm	macro_f1	unseen_locale	0.1231	0.05664	0.08496	no significant change
rules over llm	macro_f1	tier:clean	0.1981	0.01758	0.04143	improvement
rules over llm	macro_f1	tier:partial	0.2155	0.01562	0.04143	improvement
rules over llm	macro_f1	tier:mixed	0.1653	0.05664	0.08496	no significant change
rules over llm	macro_f1	tier:hostile	-0.0584	0.19531	0.23872	no significant change
rules over llm	finding_precision	MISSING_AUTOCOMPLETE	0.2197	0.00781	0.02578	improvement
rules over llm	finding_recall	MISSING_AUTOCOMPLETE	-0.1152	0.00195	0.02578	regression
rules over llm	finding_precision	WRONG_AUTOCOMPLETE	0.7680	0.00781	0.02578	improvement
rules over llm	finding_recall	WRONG_AUTOCOMPLETE	-0.0333	1.00000	1.00000	no significant change
ngram over llm	macro_f1	all	0.1041	0.08594	0.11816	no significant change
ngram over llm	macro_f1	seen_locales	0.1362	0.03516	0.06445	no significant change
ngram over llm	macro_f1	unseen_locale	-0.0409	0.37500	0.41250	no significant change
ngram over llm	macro_f1	tier:clean	0.2654	0.00391	0.02578	improvement
ngram over llm	macro_f1	tier:partial	0.2768	0.00391	0.02578	improvement
ngram over llm	macro_f1	tier:mixed	0.0799	0.30664	0.34894	no significant change
ngram over llm	macro_f1	tier:hostile	0.0143	0.79102	0.81573	no significant change
ngram over llm	finding_precision	MISSING_AUTOCOMPLETE	0.1927	0.00781	0.02578	improvement

Comparison	Metric	Slice	Effect	p	adjusted p	Verdict
ngram over llm	finding_ recall	MISSING_ AUTOCOMPLETE	-0.2722	0.00195	0.02578	regression
ngram over llm	finding_ precision	WRONG_ AUTOCOMPLETE	0.6810	0.00781	0.02578	improvement
ngram over llm	finding_ recall	WRONG_ AUTOCOMPLETE	-0.2667	0.03125	0.06066	no significant change
ngram over rules	macro_f1	all	-0.0517	0.22656	0.26702	no significant change
ngram over rules	macro_f1	seen_locales	-0.0256	0.59375	0.63206	no significant change
ngram over rules	macro_f1	unseen_locale	-0.1640	0.02148	0.04727	regression
ngram over rules	macro_f1	tier:clean	0.0673	0.02344	0.04834	improvement
ngram over rules	macro_f1	tier:partial	0.0613	0.05469	0.08496	no significant change
ngram over rules	macro_f1	tier:mixed	-0.0854	0.15820	0.20080	no significant change
ngram over rules	macro_f1	tier:hostile	0.0727	0.08203	0.11770	no significant change
ngram over rules	finding_ precision	MISSING_ AUTOCOMPLETE	-0.0269	0.01758	0.04143	regression
ngram over rules	finding_ recall	MISSING_ AUTOCOMPLETE	-0.1570	0.01758	0.04143	regression
ngram over rules	finding_ precision	WRONG_ AUTOCOMPLETE	-0.0870	0.04688	0.08141	no significant change
ngram over rules	finding_ recall	WRONG_ AUTOCOMPLETE	-0.2333	0.09375	0.12375	no significant change

Source: `experiments/results/analysis/2026-08-26T21-20-11Z_p6-analysis_6457ac7/analysis.json`

Table A.2: All comparisons in the corrected family, read in the direction they were measured. The *Comparison* column is built from the baseline, the candidate and the effect rather than from the comparison name, which reads backwards at a glance.

A.2 The locale by tier grid

Locale	Tier	Fields	rules	ngram	llm
de-DE	clean	84	0.7962	0.9612	0.6732
de-DE	hostile	104	0.3838	0.5401	0.4420
de-DE	mixed	104	0.7518	0.7561	0.5888
de-DE	partial	84	0.7962	0.9522	0.6740
en-GB	clean	88	0.8741	0.9902	0.6029
en-GB	hostile	108	0.4667	0.6258	0.5611
en-GB	mixed	108	0.7493	0.7829	0.6478
en-GB	partial	88	0.8741	0.9902	0.6826
en-NG	clean	85	0.8741	0.9902	0.7011
en-NG	hostile	104	0.4453	0.6284	0.5656
en-NG	mixed	104	0.7790	0.8753	0.6555
en-NG	partial	84	0.8741	0.9902	0.6791
en-US	clean	88	0.8741	0.9902	0.7210
en-US	hostile	108	0.4667	0.5704	0.5333
en-US	mixed	108	0.8068	0.7727	0.6267
en-US	partial	88	0.8741	0.9902	0.6251
fr-FR	clean	84	0.7962	0.6919	0.6373
fr-FR	hostile	104	0.4002	0.3347	0.4733

Locale	Tier	Fields	rules	ngram	llm
fr-FR	mixed	104	0.7366	0.5842	0.6330
fr-FR	partial	84	0.7962	0.6919	0.5720
ja-JP	clean	91	0.9074	0.9349	0.6100
ja-JP	hostile	111	0.5060	0.5125	0.4917
ja-JP	mixed	111	0.7937	0.7098	0.6259
ja-JP	partial	91	0.9074	0.9016	0.6338

Source: `experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7/benchmark.json`

Table A.4: Macro-F1 for every locale and tier cell, all three engines. Every cell clears the reporting minimum, so nothing here is suppressed.

A.3 Per label behaviour

Label	Support	rules	ngram	llm
CC_EXP_SPLIT_MONTH	48	1.0000	1.0000	0.6047
CC_EXP_SPLIT_YEAR	48	1.0000	1.0000	0.6842
COMPOSITE_UNSPPLIT	48	0.8293	0.9565	0.0000
NOT_AUTOFILLABLE	336	0.8819	0.9587	0.7147
UNKNOWN	120	0.3301	0.7255	0.2458
address-level1	64	1.0000	0.7211	0.9375
address-level2	84	0.8000	0.6900	0.8299
address-line1	24	0.6286	0.6032	0.2901
address-line2	24	0.9091	0.6364	0.7407
address-line3	24	0.8837	0.6842	0.7917
bday	24	0.8293	0.9362	0.8293
cc-csc	96	0.8000	0.8804	0.8977
cc-exp	24	0.9333	0.8837	0.5882
cc-exp-month	24	1.0000	0.8235	0.6111
cc-exp-year	24	0.7692	0.6667	0.6111
cc-name	48	0.9677	0.7547	0.8276
cc-number	96	0.8075	0.8639	0.9005
cc-type	72	1.0000	0.9714	0.5743
country	96	0.8421	0.7630	0.3051
country-name	48	0.2545	0.6250	0.4458
current-password	48	0.0000	0.8889	0.6667
email	144	0.9565	0.9329	0.9531
family-name	84	0.9091	0.7547	0.7891
given-name	84	0.9091	0.7027	0.7750
name	24	1.0000	0.8163	0.7164
new-password	96	0.7114	0.9495	0.8108
one-time-code	48	0.7692	0.6917	0.7105
organization	48	1.0000	0.7273	0.7532
postal-code	105	0.7861	0.8542	0.8646
sex	24	1.0000	0.9020	0.5882
street-address	72	0.6154	0.6142	0.3409
tel	24	0.6071	0.7200	0.6102
tel-country-code	24	1.0000	1.0000	0.8000
tel-national	24	0.0000	0.5862	0.5882
url	24	0.8571	0.8571	0.8571
username	72	0.8923	0.6949	0.8872

Source: `experiments/results/test/2026-08-26T21-15-35Z_p6-rules_6457ac7/metrics.json`

Source: experiments/results/test/2026-08-26T21-17-53Z_p6-ngram_6457ac7/metrics.json

Source: experiments/results/test/2026-08-26T20-32-48Z_p6-llm_6457ac7/metrics.json

Table A.6: F1 per observed label per engine, with the test split support beside it. Read the rows against the support column: several labels are carried by one template in one position.

A.4 Confusions

Engine	Answer key	Predicted	Fields
rules	NOT_AUTOFILLABLE	UNKNOWN	71
	current-password	UNKNOWN	48
	new-password	UNKNOWN	43
	street-address	UNKNOWN	40
	postal-code	UNKNOWN	37
	country-name	country	36
	cc-csc	UNKNOWN	32
	cc-number	UNKNOWN	31
	address-level2	UNKNOWN	28
	one-time-code	UNKNOWN	18
ngram	country	address-level1	30
	username	given-name	17
	cc-number	cc-name	13
	street-address	one-time-code	12
	address-level1	country	11
	UNKNOWN	one-time-code	10
	UNKNOWN	address-line2	9
	country-name	address-level2	9
	current-password	new-password	8
	NOT_AUTOFILLABLE	email	7
llm	UNKNOWN	NOT_AUTOFILLABLE	91
	country	country-name	76
	cc-type	NOT_AUTOFILLABLE	43
	street-address	address-line1	40
	COMPOSITE_UNSPLOT	address-line1	36
	CC_EXP_SPLIT_MONTH	cc-exp-month	22
	CC_EXP_SPLIT_YEAR	cc-exp-year	22
	new-password	current-password	20
	one-time-code	NOT_AUTOFILLABLE	15
	current-password	new-password	14

Source: experiments/results/test/2026-08-26T21-15-35Z_p6-rules_6457ac7/metrics.json

Source: experiments/results/test/2026-08-26T21-17-53Z_p6-ngram_6457ac7/metrics.json

Source: experiments/results/test/2026-08-26T20-32-48Z_p6-llm_6457ac7/metrics.json

Table A.7: The largest confusions each engine makes on the test split.

Named pair	rules	ngram	llm
address-level1 as address-level2	0	0	0
address-level2 as address-level1	0	0	2
cc-exp as cc-exp-month	0	1	4
cc-exp-month as cc-exp	0	0	0
email as username	0	0	0
tel as tel-national	0	6	0
tel-national as tel	15	7	14
username as email	0	0	0

Source: experiments/results/test/2026-08-26T21-15-35Z_p6-rules_6457ac7/metrics.json

Table A.8: The confusion pairs named in the specification before any measurement existed, and how often each actually occurred. Most of them never did, in either direction, for any engine.

A.5 Form family

Form family	Fields	rules	ngram	llm
address	352	0.7222	0.4194	0.5371
checkout	821	0.8414	0.6447	0.4741
login	216	0.6843	0.4700	0.4476
payment	384	0.8570	0.5464	0.6913
signup	544	0.7634	0.5521	0.5251

Source: experiments/results/test/2026-08-26T21-15-35Z_p6-rules_6457ac7/metrics.json

Source: experiments/results/test/2026-08-26T21-17-53Z_p6-ngram_6457ac7/metrics.json

Source: experiments/results/test/2026-08-26T20-32-48Z_p6-llm_6457ac7/metrics.json

Table A.9: Macro-F1 by form family.

A.6 Label coverage in the corpus

Label	Keyed fields in the corpus
CC_EXP_SPLIT_MONTH	120
CC_EXP_SPLIT_YEAR	120
COMPOSITE_UNSPPLIT	264
NOT_AUTOFILLABLE	1296
UNKNOWN	480
additional-name	96
address-level1	288
address-level2	404
address-line1	264
address-line2	264
address-line3	96
bday	96
cc-csc	384
cc-exp	72
cc-exp-month	96
cc-exp-year	96
cc-name	264
cc-number	384
cc-type	168
country	360
country-name	192
current-password	168
email	600
family-name	392
given-name	392
honorific-prefix	144
honorific-suffix	144
name	144
new-password	360
nickname	144
one-time-code	240
organization	192

Label	Keyed fields in the corpus
postal-code	403
sex	96
street-address	168
tel	144
tel-country-code	144
tel-extension	96
tel-national	144
transaction-amount	120
url	120
username	192

Source: `corpus/manifest.json`

Table A.11: Keyed fields per label across the whole generated corpus. This is the evidence for the growth rule’s clause that every label must be emitted by at least one answer key, and it is also the honest picture of how uneven the class distribution is.

Appendix B

Provenance

B.1 Every result file this report cites

Law 3 says that no number appears in this report unless it is reproducible from a committed file under `experiments/results/` whose manifest names a commit this repository can produce. Table B.2 is that chain, written out.

Run directory	Engine	Commit	Clean
<code>experiments/results/test/2026-08-26T21-15-35Z_p6-rules_6457ac7</code>	rules	6457ac7	yes
<code>experiments/results/test/2026-08-26T21-17-53Z_p6-ngram_6457ac7</code>	ngram	6457ac7	yes
<code>experiments/results/test/2026-08-26T20-32-48Z_p6-llm_6457ac7</code>	llm	6457ac7	yes
<code>experiments/results/analysis/2026-08-26T21-20-11Z_p6-analysis_6457ac7</code>	llm against rules against ngram	6457ac7	yes
<code>experiments/results/bench/2026-08-26T20-32-48Z_bench_6457ac7</code>	llm against rules against ngram	6457ac7	yes
<code>experiments/results/analysis/2026-08-26T02-53-59Z_analysis_336175f</code>	rules against ngram	336175f	yes
<code>experiments/results/analysis/2026-08-26T06-23-41Z_p5r-analysis_0d900ff</code>	rules against ngram	0d900ff	yes
<code>experiments/results/test/2026-08-26T02-50-54Z_rules_57d2aed</code>	rules	57d2aed	yes
<code>experiments/results/test/2026-08-26T02-52-08Z_ngram_57d2aed</code>	ngram	57d2aed	yes
<code>experiments/results/test/2026-08-26T06-18-34Z_p5r-rules_0d900ff</code>	rules	0d900ff	yes
<code>experiments/results/test/2026-08-26T06-21-03Z_p5r-ngram_0d900ff</code>	ngram	0d900ff	yes

Table B.2: Every run cited in this report, its engine, the commit its manifest names, and whether the working tree was clean when it was produced. A run marked dirty is not citable and the traceability job fails on a citation of one.

The check is not a pattern match on the shape of a citation. It walks the whole chain for every reference in every prose file and every LaTeX source: the path must exist, a manifest must sit beside it or above it, that manifest must name a commit this repository can produce an object for, and the manifest must not mark the run dirty.

That distinction was learned the hard way. The job went red on a check that had passed locally minutes earlier, because the clone had been made at the default depth of one, so it held only the

tip commit and every citation of a run taken in an earlier commit failed with a message saying the commit does not resolve in this repository. That sentence was true of the clone and false of the repository. The failure was reproduced locally before anything was changed, because a failure that is fixed without being reproduced is a failure that is guessed at.

B.2 The result files that are here and are not cited

Two earlier sets of test split runs are in this repository and nothing in this report cites them.

The first pair was taken on a corpus whose test partition held one template per family. Those results were correctly taken and are correctly reported as the first, underpowered measurement. They are not edited, not moved and not deleted, because result files are append only history and the honest description of them is what they are rather than something to be tidied away.

The second pair was taken after the corpus repair and before the language model existed. Chapter 3 cites those two for exactly one purpose, which is to show what the repair did to each engine, and Table 3.3 is that comparison with the caveat attached that the two columns are not two measurements of the same thing.

B.3 Version strings in the committed run logs

A reader looking at the older run logs will find a tool version string that does not match the release the results are associated with. That is consistent rather than wrong: those runs were taken before the version bump, the engine description records the version that was actually running, and the manifest's commit resolves to a tree in which that string is correct.

Re-running them to make the string prettier is exactly the regeneration this project forbids, so they stay, and this paragraph is the explanation.

B.4 How to reproduce any number in this report

1. Clone the repository at the commit a manifest names.
2. Regenerate the corpus from the seed recorded in the corpus manifest and the base year recorded beside it. The output is byte identical or the generator has a defect.
3. Install from the committed lock file.
4. Run the evaluation for the engine and split named in the manifest's command line, which is recorded verbatim.
5. Compare the metrics document against the committed one.

The language model engine is the one step that will not reproduce bit for bit, and that is recorded rather than papered over: the serving runtime does not guarantee determinism, temperature and seed are pinned and are recorded in the run, and the engine description says in words that determinism is not claimed.

Bibliography

- [1] Olajide Badejo. autofill-audit. https://github.com/Olajide-Badejo/Autofill_audit. This project. The corpus generator, the extractor, the three engines, the evaluation runner and every result file cited in this report.
- [2] Olajide Badejo. ML-Experiment-Triage. <https://github.com/Olajide-Badejo/ML-Experiment-Triage>. The evaluation harness consumed here as a released dependency rather than vendored. Retrieved 2026-08-25.
- [3] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1):289 to 300, 1995. The multiple comparison correction applied across this project's comparison family. DOI 10.1111/j.2517-6161.1995.tb02031.x. The DOI resolves; the publisher answered HTTP 403 to an automated request from the build machine, which is a robot policy rather than a broken link, and docs/references.md records it that way.
- [4] Chrome Developer Relations. Learn forms. <https://web.dev/learn/forms/>. Developer guidance on form design, autocomplete usage, labels and inputmode, and the source of the argument that broken autofill is a conversion problem rather than a cosmetic one. Retrieved 2026-08-25.
- [5] Chrome for Developers. Chrome for developers documentation. <https://developer.chrome.com/docs/>. Chrome's own description of how autofill selects and fills fields. Retrieved 2026-08-25.
- [6] Faker developers. Faker documentation. <https://faker.readthedocs.io/en/master/>. Locale providers and seeding. The address requested at retrieval redirected here. Retrieved 2026-08-25.
- [7] Keep a Changelog. Keep a changelog. <https://keepachangelog.com/>. The changelog format this project follows, alongside Semantic Versioning at <https://semver.org/>. Retrieved 2026-08-25.
- [8] Mozilla Developer Network. HTML attribute: autocomplete. <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Attributes/autocomplete>. Per token documentation and browser support notes. The address requested at retrieval redirected here. Retrieved 2026-08-25.
- [9] Ollama. Ollama API documentation. <https://github.com/ollama/ollama/blob/main/docs/api.md>. Model library, modelfile parameters, structured outputs via JSON schema, and the OpenAI compatible endpoint. Retrieved 2026-08-25.
- [10] ONNX. sklearn-onnx converter documentation. <https://onnx.ai/sklearn-onnx/>. Converter reference and the documented limitations of the text vectorizer converters that the export phase planned around. Retrieved 2026-08-25.

- [11] ONNX Runtime. ONNX runtime documentation. <https://onnxruntime.ai/docs/>. Python API, execution providers, threading options, dynamic quantization. Retrieved 2026-08-25.
- [12] ONNX Runtime. Quantization. <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>. Dynamic INT8 quantization for transformer encoders, read for the optional transformer phase. Retrieved 2026-08-25.
- [13] Playwright. Playwright for Python. <https://playwright.dev/python/docs/intro>. Synchronous API, frames, shadow DOM piercing, waiting strategies, browser installation. Retrieved 2026-08-25.
- [14] scikit-learn developers. scikit-learn user guide. <https://scikit-learn.org/stable/>. Text feature extraction including the `char_wb` analyser and n-gram ranges, logistic regression, and probability calibration. Retrieved 2026-08-25.
- [15] Stripe. Testing. <https://docs.stripe.com/testing>. The published test card numbers used in the payment fixtures, so that no fixture can carry a real card number. Retrieved 2026-08-25.
- [16] W3C Web Accessibility Initiative. ARIA authoring practices guide. <https://www.w3.org/WAI/ARIA/apg/>. The basis for the label association signal hierarchy the extractor uses. Retrieved 2026-08-25.
- [17] WHATWG. HTML living standard: autofill. <https://html.spec.whatwg.org/multipage/form-control-infrastructure.html#autofill>. The normative definition of the `autocomplete` attribute, the autofill field name tokens, the shipping and billing modifiers, the contact modifiers, `section-*` and the autofill anchor and expectation mantle algorithm. Retrieved 2026-08-25.